IT5511
COMPUTER NETWORKS LABORATORY

NAME : **YUVARAJ K**

REGISTER NO. : **2022506079**

DEPARTMENT : **INFORMATION TECHNOLOGY**

SUBJECT CODE : **IT5512**

SUBJECT TITLE : **COMPUTER NETWORKS LABORATORY**

DEPARTMENT OF INFORMATION TECHNOLOGY
MADRAS INSTITUTE OF TECHNOLOGY,
ANNA UNIVERSITY, CHENNAI – 600 044

August 2024 - November 2024

BONAFIDE CERTIFICATE

NAME	: YUVARAJ K
REGISTER NO.	: 2022506079
DEPARTMENT	: INFORMATION TECHNOLOGY
SUBJECT CODE	: IT5512
SUBJECT TITLE	: COMPUTER NETWORKS LABORATORY

Certified that the bonafide record of practical work done by Yuvaraj K in the laboratory with subject code IT5511 and subject title Computer Networks Laboratory during the period AUGUST 2024 – NOVEMBER 2024.

DATE:

COURSE-IN-CHARGE

Submitted for the Practical Examination held on

EXAMINER

Dr. S. Vallisree

TABLE OF CONTENTS

CYCLE 1

EXP NO.	DATE	TITLE OF THE EXPERIMENT	PAGE NO.	SIGNATURE
1	05/08/2024	STUDY OF BASIC NETWORKING COMMANDS	2	
2	12/08/2024	SOCKET PROGRAMMING USING TCP – ECHO AND CHAT	21	
3	19/08/2024	SOCKET PROGRAMMING USING UDP – ECHO AND CHAT	31	
4	31/08/2024	HTTP PROTOCOLS – GET AND POST	41	
5	02/09/2024	FILE TRANSFER PROTOCOL	53	
6	02/09/2024	DOMAIN NAME SERVER	63	
7	09/09/2024	SIMPLE MAIL TRANSFER PROTCOL	71	
8	09/09/2024	POST OFFICE PROTOCOL	79	
9	16/09/2024	PING COMMAND	83	

TABLE OF CONTENTS

CYCLE 2

EXP NO.	DATE	TITLE OF THE EXPERIMENT	PAGE NO.	SIGNATURE
1	30/09/2024	STUDY OF NS-2	90	
2	30/09/2024	NETWORK TOPOLOGY USING NS-2	95	
3	07/10/2024	WIRED NETWORK USING TCP	99	
4	14/10/2024	WIRELESS NETWORK USING UDP	103	
5	21/10/2024	PERFORMANCE ANALYSIS USING XGRAPH	111	
6	28/10/2024	RIP AND OSPF	118	
7	28/10/2024	SLAAC AND DHCP	122	

CYCLE 1

OUTPUT

1)

```
C:\Users\Yuvaraj>ipconfig /all

Windows IP Configuration

   Host Name . . . . . . . . . . . . : Yuvaraj-Laptop
   Primary Dns Suffix  . . . . . . . :
   Node Type . . . . . . . . . . . . : Unknown
   IP Routing Enabled. . . . . . . . : No
   WINS Proxy Enabled. . . . . . . . : No

Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Microsoft Wi-Fi Direct Virtual Adapter
   Physical Address. . . . . . . . . : C0-A5-E8-D3-7D-0A
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes

Wireless LAN adapter Local Area Connection* 2:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Microsoft Wi-Fi Direct Virtual Adapter #2
   Physical Address. . . . . . . . . : C2-A5-E8-D3-7D-09
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes

Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Realtek PCIe GbE Family Controller
   Physical Address. . . . . . . . . : 04-BF-1B-8C-2B-C6
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
```

```
Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
   Physical Address. . . . . . . . . : C0-A5-E8-D3-7D-09
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   IPv6 Address. . . . . . . . . . . : 2401:4900:915c:d468:5f8:a9e9:6009:3d5f(Preferred)
   Temporary IPv6 Address. . . . . . : 2401:4900:915c:d468:8146:2cd7:2bc4:f6e2(Preferred)
   Link-local IPv6 Address . . . . . : fe80::e12e:ecfb:be06:7e2b%19(Preferred)
   IPv4 Address. . . . . . . . . . . : 192.168.221.166(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Lease Obtained. . . . . . . . . . : Monday, November 25, 2024 9:52:31 AM
   Lease Expires . . . . . . . . . . : Monday, November 25, 2024 10:52:30 AM
   Default Gateway . . . . . . . . . : fe80::e8b6:cbff:fe5b:f8f7%19
                                       192.168.221.24
   DHCP Server . . . . . . . . . . . : 192.168.221.24
   DHCPv6 IAID . . . . . . . . . . . : 314615272
   DHCPv6 Client DUID. . . . . . . . : 00-01-00-01-2C-8E-17-53-04-BF-1B-8C-2B-C6
   DNS Servers . . . . . . . . . . . : 192.168.221.24
   NetBIOS over Tcpip. . . . . . . . : Enabled

Ethernet adapter Bluetooth Network Connection:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Bluetooth Device (Personal Area Network)
   Physical Address. . . . . . . . . : C0-A5-E8-D3-7D-0D
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
```

EXP NO: 1	STUDY OF BASIC NETWORKING COMMANDS
05/08/2024	

AIM

To get introduced on basic commands related to networking on Windows.

COMMANDS

1) ipconfig

Displays all current TCP/IP network configuration values and refreshes Dynamic Host Configuration Protocol (DHCP) and Domain Name System (DNS) settings. Used without parameters, ipconfig displays the IP address, subnet mask, and default gateway for all adapters.

ipconfig /all

2)

```
C:\Users\Yuvaraj>ping google.com

Pinging google.com [2404:6800:4007:823::200e] with 32 bytes of data:
Reply from 2404:6800:4007:823::200e: time=33ms
Reply from 2404:6800:4007:823::200e: time=61ms
Reply from 2404:6800:4007:823::200e: time=61ms
Reply from 2404:6800:4007:823::200e: time=30ms

Ping statistics for 2404:6800:4007:823::200e:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 30ms, Maximum = 61ms, Average = 46ms
```

3)

```
C:\Users\Yuvaraj>tracert google.com

Tracing route to google.com [2404:6800:4007:823::200e]
over a maximum of 30 hops:

  1    26 ms     3 ms     4 ms  2401:4900:915c:d468::ec
  2     *        *        *     Request timed out.
  3    51 ms    65 ms    63 ms  2401:4900:0:ff0::1
  4    82 ms    46 ms    34 ms  2401:4900:0:6f5::7
  5    81 ms    41 ms    33 ms  2401:4900:0:6f5::1
  6    64 ms     *       59 ms  2404:a800:3a00:1::7da
  7    83 ms    39 ms    32 ms  2404:a800:3a00:1::7d9
  8    68 ms    47 ms    31 ms  2404:a800::92
  9     *        *        *     Request timed out.
 10    94 ms    63 ms    72 ms  2404:6800:810b::1
 11    52 ms    60 ms    75 ms  2001:4860:0:1::5662
 12    48 ms    49 ms    35 ms  2001:4860:0:1::882a
 13    56 ms    44 ms    41 ms  2001:4860:0:1::880d
 14    76 ms    24 ms    27 ms  2001:4860:0:1::55d5
 15    59 ms    34 ms    37 ms  maa03s38-in-x0e.1e100.net [2404:6800:4007:823::200e]

Trace complete.
```

2) ping

The Ping command is used to check the destination IP address to be reached and record the results. The ping command displays whether the destination responded and how long it took to receive a reply. If there is an error in the delivery to the destination, the ping command displays an error message.

ping google.com

3) traceroute (tracert)

Traceroute command (tracert) is a utility designed for displaying the time it takes for a packet of information to travel between a local computer and a destination IP address or domain. After running a traceroute command, the results displayed are a list of the 'hops' that data packets take along their path to the designated IP address or domain.

tracert google.com

4)

```
C:\Users\Yuvaraj>pathping google.com

Tracing route to google.com [2404:6800:4007:823::200e]
over a maximum of 30 hops:
  0  Yuvaraj-Laptop [2401:4900:915c:d468:8146:2cd7:2bc4:f6e2]
  1  2401:4900:915c:d468::ec
  2     *        *        *
Computing statistics for 25 seconds...
            Source to Here   This Node/Link
Hop  RTT    Lost/Sent = Pct  Lost/Sent = Pct  Address
  0                                           Yuvaraj-Laptop [2401:4900:915c:d468:8146:2cd7:2bc4:f6e2]
                                 0/ 100 =  0%   |
  1   14ms     0/ 100 =  0%     0/ 100 =  0%  2401:4900:915c:d468::ec

Trace complete.
```

5)

```
C:\Users\Yuvaraj>arp /a

Interface: 192.168.221.166 --- 0x13
  Internet Address      Physical Address      Type
  192.168.221.24        ea-b6-cb-5b-f8-f7     dynamic
  192.168.221.255       ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

4) pathping

The pathping command is a route tracing tool that combines features of the ping and tracert commands with additional information that neither of those tools provides. The pathping command sends packets to each router on the way to a final destination over a period of time, and then computes results based on the packets returned from each hop. Since the command shows the degree of packet loss at any given router or link, it is easy to determine which routers or links might be causing network problems.

pathping google.com

5) arp – Address Resolution Protocol

The address resolution protocol (arp) is a protocol used by the Internet Protocol (IP), specifically IPv4, to map IP network addresses to the hardware addresses used by a data link protocol. The protocol operates below the network layer as a part of the interface between the OSI network and OSI link
layer. It is used when IPv4 is used over Ethernet.

The term address resolution refers to the process of finding an address of a computer in a network. The address is "resolved" using a protocol in which a piece of information is sent by a client process executing on the local computer to a server process executing on a remote computer. The information received by the server allows the server to uniquely identify the network system for which the address was required and therefore to provide the required address. The address resolution procedure is completed when the client receives a response from the server containing the required address.

arp /a

6)

```
C:\Users\Yuvaraj>hostname
Yuvaraj-Laptop
```

7)

```
C:\Users\Yuvaraj>netstat /a

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    0.0.0.0:135            Yuvaraj-Laptop:0       LISTENING
  TCP    0.0.0.0:445            Yuvaraj-Laptop:0       LISTENING
  TCP    0.0.0.0:5040           Yuvaraj-Laptop:0       LISTENING
  TCP    0.0.0.0:49664          Yuvaraj-Laptop:0       LISTENING
  TCP    0.0.0.0:49665          Yuvaraj-Laptop:0       LISTENING
  TCP    0.0.0.0:49668          Yuvaraj-Laptop:0       LISTENING
  TCP    0.0.0.0:49669          Yuvaraj-Laptop:0       LISTENING
  TCP    0.0.0.0:49676          Yuvaraj-Laptop:0       LISTENING
  TCP    0.0.0.0:49684          Yuvaraj-Laptop:0       LISTENING
  TCP    127.0.0.1:5700         Yuvaraj-Laptop:0       LISTENING
  TCP    127.0.0.1:8884         Yuvaraj-Laptop:0       LISTENING
  TCP    127.0.0.1:9010         Yuvaraj-Laptop:0       LISTENING
  TCP    127.0.0.1:9010         Yuvaraj-Laptop:62213   ESTABLISHED
  TCP    127.0.0.1:9080         Yuvaraj-Laptop:0       LISTENING
  TCP    127.0.0.1:9100         Yuvaraj-Laptop:0       LISTENING
  TCP    127.0.0.1:9100         Yuvaraj-Laptop:62207   ESTABLISHED
  TCP    127.0.0.1:9180         Yuvaraj-Laptop:0       LISTENING
  TCP    127.0.0.1:45654        Yuvaraj-Laptop:0       LISTENING
  TCP    127.0.0.1:49725        Yuvaraj-Laptop:49726   ESTABLISHED
  TCP    127.0.0.1:49726        Yuvaraj-Laptop:49725   ESTABLISHED
  TCP    127.0.0.1:49727        Yuvaraj-Laptop:49728   ESTABLISHED
  TCP    127.0.0.1:49728        Yuvaraj-Laptop:49727   ESTABLISHED
  TCP    127.0.0.1:52829        Yuvaraj-Laptop:0       LISTENING
  TCP    127.0.0.1:58141        Yuvaraj-Laptop:58142   ESTABLISHED
  TCP    127.0.0.1:58142        Yuvaraj-Laptop:58141   ESTABLISHED
  TCP    127.0.0.1:60868        Yuvaraj-Laptop:65001   ESTABLISHED
  TCP    127.0.0.1:60888        Yuvaraj-Laptop:0       LISTENING
  TCP    127.0.0.1:60888        Yuvaraj-Laptop:62205   ESTABLISHED
  TCP    127.0.0.1:60945        Yuvaraj-Laptop:0       LISTENING
  TCP    127.0.0.1:62181        Yuvaraj-Laptop:62182   ESTABLISHED
  TCP    127.0.0.1:62182        Yuvaraj-Laptop:62181   ESTABLISHED
  TCP    127.0.0.1:62205        Yuvaraj-Laptop:60888   ESTABLISHED
  TCP    127.0.0.1:62207        Yuvaraj-Laptop:9100    ESTABLISHED
  TCP    127.0.0.1:62213        Yuvaraj-Laptop:9010    ESTABLISHED
  TCP    127.0.0.1:63518        Yuvaraj-Laptop:28194   SYN_SENT
```

6) hostname

Display the hostname of the machine the command is being run on.

hostname

7) netstat

The netstat command is used to display the TCP/IP network protocol statistics and information.

netstat /a

8)

```
C:\Users\Yuvaraj>route -p PRINT
===========================================================================
Interface List
  9...c0 a5 e8 d3 7d 0a ......Microsoft Wi-Fi Direct Virtual Adapter
 18...c2 a5 e8 d3 7d 09 ......Microsoft Wi-Fi Direct Virtual Adapter #2
 14...04 bf 1b 8c 2b c6 ......Realtek PCIe GbE Family Controller
 19...c0 a5 e8 d3 7d 09 ......Intel(R) Wi-Fi 6 AX201 160MHz
  4...c0 a5 e8 d3 7d 0d ......Bluetooth Device (Personal Area Network)
  1...........................Software Loopback Interface 1
===========================================================================

IPv4 Route Table
===========================================================================
Active Routes:
Network Destination        Netmask          Gateway       Interface  Metric
          0.0.0.0          0.0.0.0   192.168.221.24  192.168.221.166     50
        127.0.0.0        255.0.0.0         On-link         127.0.0.1    331
        127.0.0.1  255.255.255.255         On-link         127.0.0.1    331
  127.255.255.255  255.255.255.255         On-link         127.0.0.1    331
    192.168.221.0    255.255.255.0         On-link   192.168.221.166    306
  192.168.221.166  255.255.255.255         On-link   192.168.221.166    306
  192.168.221.255  255.255.255.255         On-link   192.168.221.166    306
        224.0.0.0        240.0.0.0         On-link         127.0.0.1    331
        224.0.0.0        240.0.0.0         On-link   192.168.221.166    306
  255.255.255.255  255.255.255.255         On-link         127.0.0.1    331
  255.255.255.255  255.255.255.255         On-link   192.168.221.166    306
===========================================================================
Persistent Routes:
  None
```

```
IPv6 Route Table
===========================================================================
Active Routes:
 If Metric Network Destination      Gateway
 19     66 ::/0                     fe80::e8b6:cbff:fe5b:f8f7
  1    331 ::1/128                  On-link
 19     66 2401:4900:915c:d468::/64 On-link
 19    306 2401:4900:915c:d468:5f8:a9e9:6009:3d5f/128
                                    On-link
 19    306 2401:4900:915c:d468:8146:2cd7:2bc4:f6e2/128
                                    On-link
 19    306 fe80::/64                On-link
 19    306 fe80::e12e:ecfb:be06:7e2b/128
                                    On-link
  1    331 ff00::/8                 On-link
 19    306 ff00::/8                 On-link
===========================================================================
Persistent Routes:
  None
```

8) route

Command to manually configure the routes in the routing table.

route -p PRIN

9)

```
C:\Users\Yuvaraj>nslookup google.com
Server:  UnKnown
Address:  192.168.221.24

Non-authoritative answer:
Name:    google.com
Addresses:  2404:6800:4007:822::200e
          142.250.195.78
```

10)

```
C:\Users\Yuvaraj>nbtstat /n

Ethernet:
Node IpAddress: [0.0.0.0] Scope Id: []

    No names in cache

Bluetooth Network Connection:
Node IpAddress: [0.0.0.0] Scope Id: []

    No names in cache

Wi-Fi:
Node IpAddress: [192.168.221.166] Scope Id: []

                NetBIOS Local Name Table

       Name               Type         Status
    ---------------------------------------------
    YUVARAJ-LAPTOP <20>  UNIQUE      Registered
    YUVARAJ-LAPTOP <00>  UNIQUE      Registered
    WORKGROUP      <00>  GROUP       Registered

Local Area Connection* 1:
Node IpAddress: [0.0.0.0] Scope Id: []

    No names in cache

Local Area Connection* 2:
Node IpAddress: [0.0.0.0] Scope Id: []

    No names in cache
```

9) nslookup

MS-DOS utility that enables a user to look up an IP address of a domain or host on a network.

nslookup google.com

10) nbtstat

Displays NetBIOS over TCP/IP (NetBT) protocol statistics, NetBIOS name tables for both the local computer and remote computers, and the NetBIOS name cache. Nbtstat allows a refresh of the NetBIOS name cache and the names registered with Windows Internet Name Service (WINS).

nbtstat /n

11)

```
C:\Users\Yuvaraj>netsh trace show interface

Wireless LAN adapter Local Area Connection* 1:
    Description:     Microsoft Wi-Fi Direct Virtual Adapter
    Interface GUID:  {6E71C6A5-DB87-4B10-A1E8-8FA752ECB7DE}
    Interface Index: 9
    Interface Luid:  0x47008001000000

Wireless LAN adapter Local Area Connection* 2:
    Description:     Microsoft Wi-Fi Direct Virtual Adapter #2
    Interface GUID:  {F6B978A1-47C3-499F-BD1B-CBFBB4415F73}
    Interface Index: 18
    Interface Luid:  0x47008002000000

Ethernet adapter Ethernet:
    Description:     Realtek PCIe GbE Family Controller
    Interface GUID:  {E2DFFF1E-FE2E-40E0-9B90-DBBE2C2FE154}
    Interface Index: 14
    Interface Luid:  0x6008001000000

Wireless LAN adapter Wi-Fi:
    Description:     Intel(R) Wi-Fi 6 AX201 160MHz
    Interface GUID:  {FF01FE84-8089-41CC-BE9D-AF634A1378D0}
    Interface Index: 19
    Interface Luid:  0x47008000000000

Ethernet adapter Bluetooth Network Connection:
    Description:     Bluetooth Device (Personal Area Network)
    Interface GUID:  {212FFCBD-9887-4D45-9788-0AE0A86F5D80}
    Interface Index: 4
    Interface Luid:  0x6008002000000
```

12)

```
C:\Users\Yuvaraj>getmac

Physical Address    Transport Name
=================== ==========================================================
C0-A5-E8-D3-7D-09   \Device\Tcpip_{FF01FE84-8089-41CC-BE9D-AF634A1378D0}
04-BF-1B-8C-2B-C6   Media disconnected
C0-A5-E8-D3-7D-0D   Media disconnected
```

11) netsh

Netsh command is powerful utility to view and configure almost all of network adapters. When passed without arguments it lands inside a interactive shell.

netsh trace show interface

12) getmac

The getmac command provides an easy way to find the MAC address of your device. Prints more than one MAC address device has multiple network adapters.

getmac

13)

```
C:\Users\Yuvaraj>net config workstation
Computer name                        \\YUVARAJ-LAPTOP
Full Computer name                   Yuvaraj-Laptop
User name                            Yuvaraj

Workstation active on
        NetBT_Tcpip_{FF01FE84-8089-41CC-BE9D-AF634A1378D0} (C0A5E8D37D09)

Software version                     Windows 10 Home Single Language

Workstation domain                   WORKGROUP
Logon domain                         YUVARAJ-LAPTOP

COM Open Timeout (sec)               0
COM Send Count (byte)                16
COM Send Timeout (msec)              250
The command completed successfully.
```

14)

```
C:\Users\Yuvaraj>ipconfig /flushdns

Windows IP Configuration

Successfully flushed the DNS Resolver Cache.
```

13) net

The net command allows user to manage many different aspects of a network and its settings such as network shares, users and print jobs etc..

net config workstation

14) ipconfig /flushdns

This command is only needed if you’re having trouble with your networks DNS configuration.

ipconfig /flushdns

15)

```
C:\Users\Yuvaraj>systeminfo

Host Name:                 YUVARAJ-LAPTOP
OS Name:                   Microsoft Windows 11 Home Single Language
OS Version:                10.0.22631 N/A Build 22631
OS Manufacturer:           Microsoft Corporation
OS Configuration:          Standalone Workstation
OS Build Type:             Multiprocessor Free
Registered Owner:          Yuvaraj
Registered Organization:   N/A
Product ID:                00342-42655-93599-AAOEM
Original Install Date:     5/12/2024, 3:43:22 PM
System Boot Time:          11/14/2024, 12:25:30 AM
System Manufacturer:       Dell Inc.
System Model:              Dell G15 5530
System Type:               x64-based PC
Processor(s):              1 Processor(s) Installed.
                           [01]: Intel64 Family 6 Model 183 Stepping 1 GenuineIntel ~2600 Mhz
BIOS Version:              Dell Inc. 1.19.0, 8/16/2024
Windows Directory:         C:\Windows
System Directory:          C:\Windows\system32
Boot Device:               \Device\HarddiskVolume1
System Locale:             en-us;English (United States)
Input Locale:              00004009
Time Zone:                 (UTC+05:30) Chennai, Kolkata, Mumbai, New Delhi
Total Physical Memory:     16,069 MB
Available Physical Memory: 5,989 MB
Virtual Memory: Max Size:  17,106 MB
Virtual Memory: Available: 3,197 MB
Virtual Memory: In Use:    13,909 MB
Page File Location(s):     C:\pagefile.sys
Domain:                    WORKGROUP
Logon Server:              \\YUVARAJ-LAPTOP
Hotfix(s):                 4 Hotfix(s) Installed.
                           [01]: KB5045935
                           [02]: KB5027397
                           [03]: KB5046633
                           [04]: KB5044620
Network Card(s):           3 NIC(s) Installed.
                           [01]: Intel(R) Wi-Fi 6 AX201 160MHz
                                 Connection Name: Wi-Fi
                                 DHCP Enabled:    Yes
                                 DHCP Server:     192.168.221.24
                                 IP address(es)
                                 [01]: 192.168.221.166
                                 [02]: fe80::e12e:ecfb:be06:7e2b
                                 [03]: 2401:4900:915c:d468:8146:2cd7:2bc4:f6e2
                                 [04]: 2401:4900:915c:d468:5f8:a9e9:6009:3d5f
                           [02]: Realtek PCIe GbE Family Controller
                                 Connection Name: Ethernet
                                 Status:          Media disconnected
                           [03]: Bluetooth Device (Personal Area Network)
                                 Connection Name: Bluetooth Network Connection
                                 Status:          Media disconnected
Hyper-V Requirements:      A hypervisor has been detected. Features required for Hyper-V will not be displayed.
```

15) systeminfo

Systeminfo command collects all the important details of a system including processor details, model of the system and network interfaces and puts them in a readable format.

systeminfo

RESULT

Hence, some basic Windows networking commands were tested successfully.

EXP NO: 2	SOCKET PROGRAMMING USING TCP - ECHO AND CHAT
12/08/2024	

AIM

To create echo and chat applications by implementing sockets using Transmission Control Protocol (TCP) in Java.

ALGORITHM – TCP ECHO SERVER

1) Declare the variables for the socket.
2) Specify the family, protocol, IP address, and port number.
3) Create a socket using socket() function.
4) Bind IP address and port number.
5) Listen and accept the client's request for TCP connection.
6) Read and display the client's message.
7) Close the socket.

ALGORITHM – TCP ECHO CLIENT

1) Declare the variables for the socket.
2) Specify the family, protocol, IP address, and port number.
3) Create a socket using socket() function.
4) Call the connect() function for TCP.
5) Read and send the input message to the server.
6) Send 'terminate' to close the connection.

SOURCE CODE – TCP ECHO SERVER

```
import java.io.*;
import java.net.*;
public class TCPechoserver {
   public static void main(String[] args) throws IOException {
      ServerSocket ss = new ServerSocket(8080);
      Socket s = ss.accept();
      System.out.println("[INFO] : Client connected.");
```

OUTPUT – TCP ECHO SERVER

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_02>javac TCPechoserver.java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_02>java TCPechoserver
[INFO] : Client connected.
[CLIENT] : Hello!
[CLIENT] : This is the client messaging!
[CLIENT] : The client's messages are echoed in the server.
[CLIENT] : terminate
[INFO] : Client terminated connection.

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_02>
```

OUTPUT – TCP ECHO CLIENT

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_02>javac TCPechoclient.java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_02>java TCPechoclient
[INFO] : Server connection established.
[SYSTEM] > Message: Hello!
[SYSTEM] > Message: This is the client messaging!
[SYSTEM] > Message: The client's messages are echoed in the server.
[SYSTEM] > Message: Terminate
[INFO] : Connection terminated.

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_02>
```

```
        InputStreamReader socIn = new
InputStreamReader(s.getInputStream());
        BufferedReader in = new BufferedReader(socIn);
        String x;
        while (true) {
            x = in.readLine();
            System.out.println("[CLIENT] : " + x);
            if (x.equalsIgnoreCase("terminate")) {
                System.out.println("[INFO] : Client terminated connection.");
                break;
            }
        }
        ss.close();
    }
}
```

SOURCE CODE – TCP ECHO CLIENT

```
import java.io.*;
import java.net.*;
public class TCPechoclient {
    public static void main(String[] args) throws IOException {
        Socket s = new Socket("localhost", 8080);
        System.out.println("[INFO] : Server connection established.");
        PrintWriter pr = new PrintWriter(s.getOutputStream(), true);
        InputStreamReader sysIn = new InputStreamReader(System.in);
        BufferedReader br = new BufferedReader(sysIn);
        String y;
        while (true) {
            System.out.print("[SYSTEM] > Message: ");
            y = br.readLine();
            if (y.equalsIgnoreCase("terminate")) {
                System.out.println("[INFO] : Connection terminated.");
                pr.println("terminate");
                break;
            } else pr.println(y);
        }
        s.close();
    }
}
```

ALGORITHM – TCP CHAT SERVER

1) Declare the variables for the socket.
2) Specify the family, protocol, IP address, and port number.
3) Create a socket using socket() function.
4) Bind IP address and port number.
5) Listen and accept the client's request for TCP connection.
6) Read the client's message.
7) Display the client's message, and reply.
8) Close the socket after replying with or reading 'terminate'.

ALGORITHM – TCP CHAT CLIENT

1) Declare the variables for the socket.
2) Specify the family, protocol, IP address, and port number.
3) Create a socket using socket() function.
4) Call the connect() function for TCP.
5) Read the input message.
6) Send the input message to the server.
7) Display the server's chat.
8) Send 'terminate' to close the connection.

SOURCE CODE – TCP CHAT SERVER

```
import java.io.*;
import java.net.*;
public class TCPchatserver {
   public static void main(String[] args) throws IOException {
      ServerSocket ss = new ServerSocket(8080);
      Socket s = ss.accept();
      System.out.println("[INFO] : Client connected.");
      InputStreamReader socIn = new InputStreamReader(s.getInputStream());
      BufferedReader in = new BufferedReader(socIn);
      InputStreamReader sysIn = new InputStreamReader(System.in);
      BufferedReader br = new BufferedReader(sysIn);
      PrintWriter pr = new PrintWriter(s.getOutputStream(), true);
      String x, y;
      System.out.println("[INFO] : Enter 'terminate' to end connection.");
      while (true) {
         x = in.readLine();
```

```
            if (x.equalsIgnoreCase("terminate")) {
                System.out.println("[INFO] : Client terminated connection.");
                break;
            }
            System.out.println("[CLIENT] : " + x);
            System.out.print("[SYSTEM] > Message: ");
            y = br.readLine();
            if (y.equalsIgnoreCase("terminate")) {
                System.out.print("[INFO] : Connection terminated.");
                pr.println("terminate");
                break;
            } else
                pr.println(y);
        }
        ss.close();
    }
}
```

SOURCE CODE – TCP CHAT CLIENT

```
import java.io.*;
import java.net.*;
public class TCPchatclient {
    public static void main(String[] args) throws IOException {
        Socket s = new Socket("localhost", 8080);
        System.out.println("[INFO] : Server connection established.");
        PrintWriter pr = new PrintWriter(s.getOutputStream(), true);
        InputStreamReader socIn = new InputStreamReader(s.getInputStream());
        BufferedReader in = new BufferedReader(socIn);
        InputStreamReader sysIn = new InputStreamReader(System.in);
        BufferedReader br = new BufferedReader(sysIn);
        String x, y;
        System.out.println("[INFO] : Enter 'terminate' to end connection.");
        while (true) {
            System.out.print("[SYSTEM] > Message: ");
            y = br.readLine();
            pr.println(y);
            if (y.equalsIgnoreCase("terminate")) {
                System.out.println("[INFO] : Connection terminated.");
                break;
```

OUTPUT – TCP CHAT SERVER

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_02>javac TCPchatserver.java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_02>java TCPchatserver
[INFO] : Client connected.
[INFO] : Enter 'terminate' to end connection.
[CLIENT] : This message is from the client.
[SYSTEM] > Message: This message is from the server.
[CLIENT] : This is the chat application.
[SYSTEM] > Message: Either the client or the server can terminate.
[CLIENT] : Let's make the server terminate.
[SYSTEM] > Message: Terminate
[INFO] : Connection terminated.
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_02>
```

OUTPUT – TCP CHAT CLIENT

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_02>javac TCPchatclient.java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_02>java TCPchatclient
[INFO] : Server connection established.
[INFO] : Enter 'terminate' to end connection.
[SYSTEM] > Message: This message is from the client.
[SERVER] : This message is from the server.
[SYSTEM] > Message: This is the chat application.
[SERVER] : Either the client or the server can terminate.
[SYSTEM] > Message: Let's make the server terminate.
[INFO] : Server terminated connection.

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_02>
```

```
            }
            x = in.readLine();
            if (x.equalsIgnoreCase("terminate")) {
                System.out.println("[INFO] : Server terminated connection.");
                break;
            }
            System.out.println("[SERVER] : " + x);
        }
        s.close();
    }
}
```

RESULT

Hence, echo and chat applications were created and executed successfully with TCP using Java.

EXP NO: 3	SOCKET PROGRAMMING USING UDP - ECHO AND CHAT
19/08/2024	

AIM

To create echo and chat applications using User Datagram Protocol (UDP) in Java.

ALGORITHM – UDP ECHO SERVER

1) Declare the variables for the socket.
2) Specify the family, protocol, IP address, and port number.
3) Create a socket using socket() function.
4) Bind IP address and port number.
5) Listen and accept the client's request for UDP connection.
6) Read and display the client's message.
7) Close the socket.

ALGORITHM – UDP ECHO CLIENT

1) Declare the variables for the socket.
2) Specify the family, protocol, IP address, and port number.
3) Create a socket using socket() function.
4) Call the connect() function for UDP.
5) Read and send the input message to the server.
6) Send 'terminate' to close the connection.

SOURCE CODE – UDP ECHO SERVER

```
import java.io.*;
import java.net.*;
public class UDPechoserver {
   public static void main(String[] args) throws IOException {
      DatagramSocket server = new DatagramSocket(4160);
      byte[] buf = new byte[1024];
      DatagramPacket packet = new DatagramPacket(buf, buf.length);
      System.out.println("[INFO] : Server started.");
```

OUTPUT – UDP ECHO SERVER

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_03>javac UDPechoserver.java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_03>java UDPechoserver
[INFO] : Server started.
[CLIENT] : This is UDP Echo.
[CLIENT] : The client is messaging.
[CLIENT] : The server is echoing its messages.
[INFO] : Client terminated connection.

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_03>
```

OUTPUT – UDP ECHO CLIENT

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_03>javac UDPechoclient.java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_03>java UDPechoclient
[INFO] : Client started.
[INFO] : Enter 'terminate' to end connection.
[SYSTEM] > Message: This is UDP Echo.
[SYSTEM] > Message: The client is messaging.
[SYSTEM] > Message: The server is echoing its messages.
[SYSTEM] > Message: Terminate
[INFO] : Connection terminated.

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_03>
```

```
        while (true) {
            server.receive(packet);
            String x = new String(packet.getData(), 0, packet.getLength());
            if (x.equalsIgnoreCase("terminate")) {
                System.out.println("[INFO] : Client terminated connection.");
                break;
            }
            System.out.println("[CLIENT] : " + x);
        }
        server.close();
    }
}
```

SOURCE CODE – UDP ECHO CLIENT

```
import java.io.*;
import java.net.*;
public class UDPechoclient {
    public static void main(String[] args) throws IOException {
        DatagramSocket client = new DatagramSocket();
        InetAddress add = InetAddress.getByName("localhost");
        byte buf[] = new byte[1024];
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        System.out.println("[INFO] : Client started.\n[INFO] : Enter 'terminate' to
end connection.");
        while (true) {
            System.out.print("[SYSTEM] > Message: ");
            String x = new String(br.readLine());
            buf = x.getBytes();
            if (x.equalsIgnoreCase("terminate")) {
                System.out.println("[INFO] : Connection terminated.");
                client.send(new DatagramPacket(buf, x.length(), add, 4160));
                break;
            }
            client.send(new DatagramPacket(buf, x.length(), add, 4160));
        }
        client.close();
    }
}
```

ALGORITHM – UDP CHAT SERVER

1) Declare the variables for the socket.
2) Specify the family, protocol, IP address, and port number.
3) Create a socket using socket() function.
4) Bind IP address and port number.
5) Listen and accept the client's request for UDP connection.
6) Read the client's message.
7) Display the client's message, and reply.
8) Close the socket after replying with or reading 'terminate'.

ALGORITHM – UDP CHAT CLIENT

1) Declare the variables for the socket.
2) Specify the family, protocol, IP address, and port number.
3) Create a socket using socket() function.
4) Call the connect() function for UDP.
5) Read the input message.
6) Send the input message to the server.
7) Display the server's chat.
8) Send 'terminate' to close the connection.

SOURCE CODE – UDP CHAT SERVER

```
import java.io.*;
import java.net.*;
public class UDPchatserver {
   public static void main(String[] args) throws IOException {
      int c_port = 4150, s_port = 4160;
      DatagramSocket server = new DatagramSocket(s_port);
      byte[] buf = new byte[1024];
      DatagramPacket packet = new DatagramPacket(buf, buf.length);
      BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
      InetAddress ia = InetAddress.getLocalHost();
      System.out.println("[INFO] : Server started.\n[INFO] : Enter 'terminate' to
end connection.");
      while (true) {
         server.receive(packet);
         String x = new String(packet.getData(), 0, packet.getLength());
```

```
            if (x.equalsIgnoreCase("terminate")) {
                System.out.println("[INFO] : Client terminated connection.");
                break;
            }
            System.out.println("[CLIENT] : " + x);
            System.out.print("[SYSTEM] > Message: ");
            String y = br.readLine();
            buf = y.getBytes();
            server.send(new DatagramPacket(buf, y.length(), ia, c_port));
            if (y.equalsIgnoreCase("terminate")) {
                System.out.println("[INFO] : Connection terminated.");
                break;
            }
        }
        server.close();
    }
}
```

SOURCE CODE – UPD CHAT CLIENT

```
import java.io.*;
import java.net.*;
public class UDPchatclient {
    public static void main(String[] args) throws IOException {
        int c_port = 4150, s_port = 4160;
        DatagramSocket client = new DatagramSocket(c_port);
        InetAddress add = InetAddress.getByName("localhost");
        byte buf[] = new byte[1024];
        DatagramPacket dp = new DatagramPacket(buf, buf.length);
        BufferedReader br = new BufferedReader(new
InputStreamReader(System.in));
        System.out.println("[INFO] : Client started.\n[INFO] : Enter 'terminate' to
end connection.");
        while (true) {
            System.err.print("[SYSTEM] > Message: ");
            String x = br.readLine();
            buf = x.getBytes();
            if (x.equalsIgnoreCase("terminate")) {
                System.out.println("[INFO] : Connection Terminated.");
                client.send(new DatagramPacket(buf, x.length(), add, s_port));
```

OUTPUT – UDP CHAT SERVER

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_03>javac UDPchatserver.java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_03>java UDPchatserver
[INFO] : Server started.
[INFO] : Enter 'terminate' to end connection.
[CLIENT] : This is UDP Chat application.
[SYSTEM] > Message: Both the server and the client are communicating.
[CLIENT] : This message is from the client.
[SYSTEM] > Message: This message is from the server.
[CLIENT] : Either one can terminate.
[SYSTEM] > Message: Terminate
[INFO] : Connection terminated.

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_03>
```

OUTPUT – UDP CHAT CLIENT

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_03>javac UDPchatclient.java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_03>java UDPchatclient
[INFO] : Client started.
[INFO] : Enter 'terminate' to end connection.
[SYSTEM] > Message: This is UDP Chat application.
[SERVER] : Both the server and the client are communicating.
[SYSTEM] > Message: This message is from the client.
[SERVER] : This message is from the server.
[SYSTEM] > Message: Either one can terminate.
[INFO] : Server terminated connection.

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_03>
```

```
            break;
        }
        client.send(new DatagramPacket(buf, x.length(), add, s_port));
        client.receive(dp);
        String y = new String(dp.getData(), 0, dp.getLength());
        if (y.equalsIgnoreCase("terminate")){
            System.out.println("[INFO] : Server terminated connection.");
            break;
        }
        System.out.println("[SERVER] : " + y);
    }
    client.close();
  }
}
```

RESULT

Hence, echo and chat applications were created and executed successfully with UDP using Java.

EXP NO: 4	HTTP PROTOCOLS – GET AND POST
31/08/2024	

AIM

To create Java programs that implement HTTP protocol using GET and POST.

ALGORITHM – HTTP GET SERVER

1) Declare the variables for the socket.
2) Create a socket using socket() function.
3) Listen and accept the client's request for connection.
4) Read and display the client's message, which will be a URL.
5) Establish GET HTTP connection for the URL.
6) Set request property to USER_AGENT.
7) Get the response code; if it is okay, go to step 8, otherwise raise an exception.
8) Accept the message from the URL by using HTTP connection in a buffer.
9) Convert the buffered message to String format and print it.
10) Send this message back to the client.

ALGORITHM – HTTP GET CLIENT

1) Create a client socket with port 6789.
2) Declare the variable for the socket.
3) Accept the URL from the user.
4) Write the URL to the socket to send it to the server.
5) Read the GET response.

SOURCE CODE – HTTP GET SERVER

```
import java.io.*;
import java.net.*;
public class HTTPGetServer {
   private static final String USER_AGENT = "Google Chrome";
   static String sendGET(String GET_URL) throws Exception {
      URL URLObj = new URL(GET_URL);
```

```
        HttpURLConnection con = (HttpURLConnection)
URLObj.openConnection();
        con.setRequestMethod("GET");
        con.setRequestProperty("User-Agent", USER_AGENT);
        int responseCode = con.getResponseCode();
        System.out.println("[INFO] : Response Code: " + responseCode);
        if (responseCode == HttpURLConnection.HTTP_OK) {
            BufferedReader in = new BufferedReader(new
InputStreamReader(con.getInputStream()));
            String inputLine;
            StringBuffer response = new StringBuffer();
            while ((inputLine = in.readLine()) != null) {
                response.append(inputLine);
            }
            in.close();
            return(response.toString());
        } else {
            System.out.println("[ERROR] : GET request failed.");
            return (null);
        }
    }
    public static void main(String[] args) throws Exception {
        ServerSocket ss=new ServerSocket(6789);
        while(true){
            Socket conSoc= ss.accept();
            BufferedReader ifc = new BufferedReader(new
InputStreamReader(conSoc.getInputStream()));
            DataOutputStream otc = new
DataOutputStream(conSoc.getOutputStream());
            String cs=ifc.readLine()+'\n';
            System.out.println("[INFO] : Requested URL: "+cs);
            String GET_URL = cs;
            otc.writeBytes(sendGET(GET_URL)+'\n');
            System.out.println("[INFO] : GET Request successful.");
            break;
        }
        ss.close();
    }
}
```

OUTPUT – HTTP GET SERVER

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_04>java HTTPGetServer
[INFO] : Requested URL: http://localhost/CN/get.php?n=Yuvaraj

[INFO] : Response Code: 200
[INFO] : GET Request successful.

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_04>
```

OUTPUT – HTTP GET CLIENT

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_04>java HTTPGetClient
[SYSTEM] > Enter URL: http://localhost/CN/get.php?n=Yuvaraj
[SERVER] : GET Response:
<html><body>    <h1>HTTP GET</h1>       <form method="GET">               <label for="n">Name:
</label>            <input type="text" name="n" required>               <br><br>
<input type="submit" value="Submit"></form> Name: Yuvaraj</body></html>

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_04>
```

SOURCE CODE – HTTP GET CLIENT

```
import java.io.*;
import java.net.*;
public class HTTPGetClient {
    public static void main(String[] args) throws Exception {
        BufferedReader ifu = new BufferedReader(new
InputStreamReader(System.in));
        Socket clientSocket = new Socket("localhost", 6789);
        DataOutputStream ots = new
DataOutputStream(clientSocket.getOutputStream());
        BufferedReader ifs = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
        System.out.print("[SYSTEM] > Enter URL: ");
        String sentence = ifu.readLine();
        ots.writeBytes(sentence+'\n');
        String ms = ifs.readLine();
        System.out.println("[SERVER] : GET Response:\n"+ms);
        clientSocket.close();
    }
}
```

SOURCE CODE – get.php

```
<html>
<body>
    <h1>HTTP GET</h1>
    <form method="GET">
        <label for="n">Name: </label>
        <input type="text" name="n" required>
        <br><br>
        <input type="submit" value="Submit">
    </form>
    <?php
        if (isset($_GET["n"])) {
            echo "Name: ".$_GET["n"];
        }
    ?>
</body>
</html>
```

ALGORITHM – HTTP POST SERVER

1) Declare the variables for the socket.
2) Create a socket using socket() function.
3) Listen and accept the client's request for connection.
4) Read and display the client's message, which will be a URL.
5) Establish POST HTTP connection for the URL.
6) Set request property to USER_AGENT.
7) Get the response code; if it is okay, go to step 8, otherwise raise an exception.
8) Accept the message from the URL by using HTTP connection in a buffer.
9) Convert the buffered message to String format and print it.
10) Send this message back to the client.

ALGORITHM – HTTP POST CLIENT

1) Create a client socket with port 6789.
2) Declare the variable for the socket.
3) Accept the URL from the user.
4) Write the URL to the socket to send it to the server.
5) Read the POST response.

SOURCE CODE – HTTP POST SERVER

```
import java.io.*;
import java.net.*;
public class HTTPPostServer {
   private static final String USER_AGENT = "Google Chrome";
   static String sendPOST(String POST_URL) throws IOException {
      URL obj = new URL(POST_URL);
      HttpURLConnection con = (HttpURLConnection)
      obj.openConnection();
      con.setRequestMethod("POST");
      con.setRequestProperty("User-Agent", USER_AGENT);
      con.setDoOutput(true);
      int responseCode = con.getResponseCode();
      System.out.println("[INFO] : Response Code: " + responseCode);
      if (responseCode == HttpURLConnection.HTTP_OK) {
         BufferedReader in = new BufferedReader(new
InputStreamReader(con.getInputStream()));
         String inputLine;
```

```
            StringBuffer response = new StringBuffer();
            while ((inputLine = in.readLine()) != null) {
                response.append(inputLine);
            }
            in.close(); return(response.toString());
        } else  {
            System.out.println("[ERROR] : POST Request failed."); return(null);
        }
    }
    public static void main(String a[]) throws Exception {
        ServerSocket ss=new ServerSocket(6789);
        while(true) {
            Socket consoc= ss.accept();
            BufferedReader ifc =new BufferedReader(new
InputStreamReader(consoc.getInputStream()));
            DataOutputStream otc =new
DataOutputStream(consoc.getOutputStream());
            String ps=ifc.readLine()+'\n';
            System.out.println("[INFO] : Requested URL: "+ps);
            String POST_URL = ps;
otc.writeBytes(sendPOST(POST_URL)+'\n');
            System.out.println("[INFO] : POST Request successful.");
            break;
        }
        ss.close();
    }
}
```

SOURCE CODE – HTTP POST CLIENT

```
import java.io.*;
import java.net.*;
public class HTTPPostClient {
    public static void main(String a[]) throws Exception {
        BufferedReader ifu = new BufferedReader(new
InputStreamReader(System.in));
        Socket clientSocket = new Socket("localhost",6789);
        DataOutputStream ots = new
DataOutputStream(clientSocket.getOutputStream());
```

OUTPUT – HTTP POST SERVER

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_04>java HTTPPostServer
[INFO] : Requested URL: http://localhost/CN/post.php?n=Yuvaraj

[INFO] : Response Code: 200
[INFO] : POST Request successful.

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_04>
```

OUTPUT – HTTP POST CLIENT

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_04>java HTTPPostClient
[SYSTEM] > Enter URL: http://localhost/CN/post.php?n=Yuvaraj
[SERVER] : POST Response:
<html><body>    <h1>HTTP POST</h1>      <form method="POST">                <label
for="n">Name: </label>            <input type="text" name="n" required><br><br>
    <input type="submit" value="Submit">    </form> </body></html>

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_04>
```

```
        BufferedReader ifs = new BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
        System.out.print("[SYSTEM] > Enter URL: ");
        String sentence = ifu.readLine();
        ots.writeBytes(sentence+'\n');
        String ms = ifs.readLine();
        System.out.println("[SERVER] : POST Response:\n"+ms);
        clientSocket.close();
    }
}
```

SOURCE CODE – post.php

```
<html>
<body>
    <h1>HTTP POST</h1>
    <form method="POST">

        <label for="n">Name: </label>
        <input type="text" name="n" required>
        <br><br>
        <input type="submit" value="Submit">
    </form>
    <?php
        if (isset($_POST["n"])) {
            echo "Name: ".$_POST["n"];
        }
    ?>
</body>
</html>
```

RESULT

Hence, Java programs implementing HTTP GET and POST protocols were created and executed successfully.

EXP NO: 5	FILE TRANSFER PROTOCOL
02/09/2024	

AIM

To create programs in Java that implement File Transfer Protocol (FTP) by facilitating the transfer of a file between a server and a client.

ALGORITHM – FTP SERVER

1) Create the server socket.
2) Call the I/O stream.
3) Transfer the file, and print a corresponding message.
4) Send the intimation to the client.

ALGORITHM – FTP CLIENT

1) Create the client packet.
2) Send the FTP request.
3) After transferring the packets, display a corresponding statement.

SOURCE CODE – FTP SERVER

```
import java.net.*;
import java.io.*;
public class FTPServer {
    Socket ss;
    ServerSocket serverSoc;
    DataInputStream din;
    DataOutputStream dout;
    String fileName;
    File fileObject;
    public FTPServer() throws Exception {
        serverSoc = new ServerSocket(6060);
        ss = serverSoc.accept();
        System.out.println("[INFO] : Server started, socket connected to server
from "+ss.getInetAddress());
```

```
        din = new DataInputStream(ss.getInputStream());
        dout = new DataOutputStream(ss.getOutputStream());
    }
    public void startTransfer() throws Exception {
        int readByte;
        FileInputStream fStream = new FileInputStream(fileName);
        while (din.readUTF().equals("0")) continue;
        if (din.readUTF().equals("-1")) {
            fStream.close();
            return;
        }
        System.out.println("[INFO] : Requested file tranfer to
"+ss.getInetAddress()+" starting.");
        while((readByte = fStream.read()) != -1)
dout.writeUTF(String.valueOf(readByte));
        dout.writeUTF("-1");
        System.out.println("[INFO] : Requested file tranfer to
"+ss.getInetAddress()+" completed successfully.");
        fStream.close();
    }
    public void sendFile() throws Exception {
        fileName = din.readUTF();
        System.out.println("[INFO] : Requested file name: "+fileName);
        fileObject = new File(fileName);
        if(fileObject.exists()) {
            dout.writeUTF("1");
            this.startTransfer();
        } else {
            System.out.println("[ERROR] : Requested file not found.");
            dout.writeUTF("0");
        }
    }
    public static void main(String[] args) throws Exception {
        FTPServer instance = new FTPServer();
        instance.sendFile();
        instance.ss.close();
        instance.serverSoc.close();
        System.out.println("[INFO] : Socket disconnected and server closed.");
    }
}
```

SOURCE CODE – FTP CLIENT

```
import java.io.*;
import java.net.*;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.Scanner;;
public class FTPClient {
   Socket soc;
   DataInputStream din;
   DataOutputStream dout;
   String fileName;
   public FTPClient() throws Exception {
      soc = new Socket("127.0.0.1", 6060);
      System.out.println("[INFO] : Socket connected at "+soc.getInetAddress());
      din = new DataInputStream(soc.getInputStream());
      dout = new DataOutputStream(soc.getOutputStream());
   }
   public void getFile() throws Exception {
      String fileIn;
      Scanner in = new Scanner(System.in);
      System.out.print("[SYSTEM] > Enter file name to download: ");
      fileName = in.nextLine();
      dout.writeUTF(fileName);
      // fileName consists of the whole path, we only need the actual name of the
file while transferring and saving it, so we use Path for that purpose.
      Path file = Paths.get(fileName);
      fileName = file.getFileName().toString();
      if(din.readUTF().equals("1")) {
         System.out.println("[INFO] : The requested file is available at
"+soc.getInetAddress());
         File fileObj = new File(fileName);
         if(fileObj.exists()) {
            System.out.println("[INFO] : The requested file already exists in
current folder.");
            dout.writeUTF("0") ;
            System.out.print("[SYSTEM] > Do you want to replace the
file[Y/N]?: ");
            String ch = in.nextLine();
```

OUTPUT – FTP SERVER

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_05>java FTPServer
[INFO] : Server started, socket connected to server from /127.0.0.1
[INFO] : Requested file name: sample.txt
[INFO] : Socket disconnected and server closed.

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_05>
```

OUTPUT – FTP CLIENT

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_05>java FTPClient
[INFO] : Socket connected at /127.0.0.1
[SYSTEM] > Enter file name to download: sample.txt
[INFO] : The requested file is available at /127.0.0.1
[INFO] : The requested file already exists in current folder.
[SYSTEM] > Do you want to replace the file[Y/N]?: N
[SYSTEM] > Do you want to rename the received file[Y/N]?: N
[INFO] : File already exists. No further operations done.
[INFO] : Socket closed.

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_05>
```

```
            if(ch.equalsIgnoreCase("n")) {
                System.out.print("[SYSTEM] > Do you want to rename the
received file[Y/N]?: ");
                ch = in.nextLine();
                if(ch.equalsIgnoreCase("n")) {
                    System.out.println("[INFO] : File already exists. No further
operations done.");
                    dout.writeUTF("-1");
                    dout.writeUTF("-1");
                    in.close();
                    return;
                }
                System.out.print("[SYSTEM] > Enter new file name: ");
                fileName = in.nextLine();
            }
        }
        dout.writeUTF("1");
        System.out.println("[INFO] : File transfer starting from
"+soc.getInetAddress());
        FileOutputStream fout = new FileOutputStream(fileName);
        while(!( fileIn = din.readUTF()).equals("-1"))
fout.write(Integer.parseInt(fileIn));
        System.out.println("[INFO] : File transfer completed from
"+soc.getInetAddress());
        fout.close();
    } else {
        System.out.println("[ERROR] : The requested file was not found on the
server.");
        System.out.println("[ERROR] : Check file name or try again.");
    }
    in.close();
  }
  public static void main(String[] args) throws Exception {
    FTPClient instance = new FTPClient();
    instance.getFile();
    instance.soc.close();
    System.out.println("[INFO] : Socket closed.");
  }
}
```

RESULT

Hence, FTP was implemented using Java, and the transfer of the requested file was performed successfully.

EXP NO: 6	DOMAIN NAME SERVER
02/09/2024	

AIM

To create a program in Java that implements Domain Name System (DNS) Server by facilitating name/IP resolution between a server and a client.

ALGORITHM

1) Enter the system name after the connection with the server is established.
2) Call the subroutine resolver and pass the system name to the resolver.
3) Open the host files in the DNS Server.
4) Check the system name with the name stored in the host file until the end of the file.
5) If the name matches, fetch the corresponding IP address and display it.
6) If no match is found, display a corresponding message.

SOURCE CODE – MANUAL INPUT RESOLVING – SERVER SIDE

```
import java.net.*;
class DNSServer1 {
   public static void main(String args[]) throws Exception {
      DatagramSocket server = new DatagramSocket(1309);
      System.out.println("[INFO] : Server started and waiting for requests.");
      byte[] sendbyte = new byte[1024]; byte[] receivebyte = new byte[1024];
      DatagramPacket receiver = new DatagramPacket(receivebyte,
receivebyte.length);
      server.receive(receiver);
      String str = new String(receiver.getData()); String s = str.trim();
      System.out.println("[INFO] : Request received to resolve "+s);
      InetAddress addr = receiver.getAddress(); int port = receiver.getPort();
      String ip[] = { "165.165.80.80", "165.165.79.1" };
      String name[] = { "www.aptitudesource.com", "www.sharifguys.com" };
      DatagramPacket sender = new DatagramPacket("empty".getBytes(),
"empty".getBytes().length, addr, port);
      for (int i = 0; i < ip.length; i++) {
```

OUTPUT – MANUAL INPUT RESOLVING – SERVER SIDE

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>java DNSServer1
[INFO] : Server started and waiting for requests.
[INFO] : Request received to resolve 165.165.80.80

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>java DNSServer1
[INFO] : Server started and waiting for requests.
[INFO] : Request received to resolve www.aptitudesource.com

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>java DNSServer1
[INFO] : Server started and waiting for requests.
[INFO] : Request received to resolve 165.165.79.1

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>java DNSServer1
[INFO] : Server started and waiting for requests.
[INFO] : Request received to resolve www.sharifguys.com

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>
```

OUTPUT – MANUAL INPUT RESOLVING – CLIENT SIDE

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>java DNSClient1
[SYSTEM] > Enter domain name or IP address: 165.165.80.80
[SERVER] : The input resolves to: www.aptitudesource.com

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>java DNSClient1
[SYSTEM] > Enter domain name or IP address: www.aptitudesource.com
[SERVER] : The input resolves to: 165.165.80.80

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>java DNSClient1
[SYSTEM] > Enter domain name or IP address: 165.165.79.1
[SERVER] : The input resolves to: www.sharifguys.com

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>java DNSClient1
[SYSTEM] > Enter domain name or IP address: www.sharifguys.com
[SERVER] : The input resolves to: 165.165.79.1

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>
```

```
            if (s.equals(ip[i])) {
                sendbyte = name[i].getBytes();
                sender = new DatagramPacket(sendbyte, sendbyte.length, addr, port);
                break;
            } else if (s.equals(name[i])) {
                sendbyte = ip[i].getBytes();
                sender = new DatagramPacket(sendbyte, sendbyte.length, addr, port);
                break;
            }
        }
        server.send(sender);
        server.close();
    }
}
```

SOURCE CODE – MANUAL INPUT RESOLVING – CLIENT SIDE

```
import java.io.*;
import java.net.*;
class DNSClient1 {
    public static void main(String args[]) throws Exception {
        DatagramSocket client = new DatagramSocket();
        InetAddress addr = InetAddress.getByName("127.0.0.1");
        byte[] sendbyte = new byte[1024];
        byte[] receivebyte = new byte[1024];
        BufferedReader in = new BufferedReader(new
InputStreamReader(System.in));
        System.out.print("[SYSTEM] > Enter domain name or IP address: ");
        sendbyte = in.readLine().getBytes();
        DatagramPacket sender = new DatagramPacket(sendbyte, sendbyte.length,
addr, 1309); client.send(sender);
        DatagramPacket receiver = new DatagramPacket(receivebyte,
receivebyte.length);
        client.receive(receiver);
        String s = new String(receiver.getData());
        if(s.isEmpty()) System.out.println("[SERVER] : The requested address or
domain doesn't exist.");
        else System.out.println("[SERVER] : The input resolves to: " + s.trim());
        client.close();
    }}
```

SOURCE CODE – DNS IMPLEMENTATION – SERVER SIDE

```
import java.net.*;
class DNSServer2 {
   public static void main(String args[]) throws Exception {
      DatagramSocket server = new DatagramSocket(1309);
      System.out.println("[INFO] : Server started and waiting for requests.");
      byte[] sendbyte = new byte[1024];
      byte[] receivebyte = new byte[1024];
      DatagramPacket receiver = new DatagramPacket(receivebyte,
receivebyte.length);
      server.receive(receiver);
      String str = new String(receiver.getData());
      String s = str.trim();
      System.out.println("[INFO] : Request received to resolve "+s);
      InetAddress addr = receiver.getAddress();
      int port = receiver.getPort();
      InetAddress IP = InetAddress.getByName(s);
      sendbyte = IP.toString().getBytes();
      DatagramPacket sender = new DatagramPacket(sendbyte, sendbyte.length,
addr, port);
      server.send(sender);
      server.close();
   }
}
```

SOURCE CODE – DNS IMPLEMENTATION – CLIENT SIDE

```
import java.io.*;
import java.net.*;
class DNSClient2 {
   public static void main(String args[]) throws Exception {
      DatagramSocket client = new DatagramSocket();
      InetAddress addr = InetAddress.getByName("127.0.0.1");
      byte[] sendbyte = new byte[1024];
      byte[] receivebyte = new byte[1024];
      BufferedReader in = new BufferedReader(new
InputStreamReader(System.in));
      System.out.print("[SYSTEM] > Enter Domain name: ");
      sendbyte = in.readLine().getBytes();
```

OUTPUT – DNS IMPLEMENTATION – SERVER SIDE

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>javac DNSServer2.java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>java DNSServer2
[INFO] : Server started and waiting for requests.
[INFO] : Request received to resolve google.com

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>
```

OUTPUT – DNS IMPLEMENTATION – CLIENT SIDE

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>javac DNSClient2.java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>java DNSClient2
[SYSTEM] > Enter Domain name: google.com
[SERVER] : The input resolves to: google.com/142.250.196.174

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_06>
```

```
        DatagramPacket sender = new DatagramPacket(sendbyte, sendbyte.length,
addr, 1309);
        client.send(sender);
        DatagramPacket receiver = new DatagramPacket(receivebyte,
receivebyte.length);
        client.receive(receiver);
        String s = new String(receiver.getData());
        if(s.isEmpty())
            System.out.println("[SERVER] : The requested address or domain
doesn't exist.");
        else
            System.out.println("[SERVER] : The input resolves to: " + s.trim());
        client.close();
    }
}
```

RESULT

Hence, DNS was successfully implemented as Java programs.

EXP NO: 7	SIMPLE MAIL TRANSFER PROTOCOL
09/09/2024	

AIM

To create programs in Java that implement Simple Mail Transfer Protocol (SMTP) by facilitating the sending of a message through a mail server.

ALGORITHM

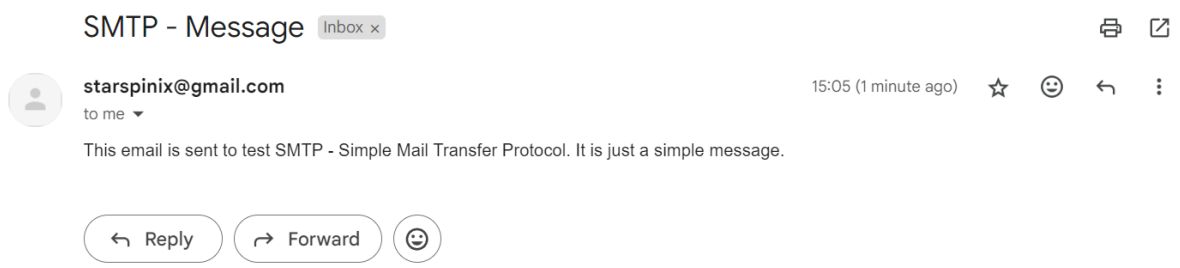
1) Send sender's mail address and password.
2) Get system properties and put host, port, SSL enable to the property object.
3) Ask the receiver's email address.
4) Get the session object for the processed password authentication purpose.
5) Create a default MIME message object to receive the message from the sender.
6) Ask the sender the subject and body of the message.
7) Send the message and display the result.

SOURCE CODE – SIMPLE MESSAGE THROUGH SMTP

```
import javax.mail.*;
import javax.mail.internet.*;
import java.util.Properties;
import java.util.Scanner;
import javax.activation.*;

public class SMTPMessage {
   public static void main(String[] args) {
      Scanner input = new Scanner(System.in);
      String SenderUser = "Starspinix";
      String SenderMail = "starspinix@gmail.com";
      System.out.print("Enter Password for Authentication: ");
      String SenderPassword = input.nextLine();
      System.out.print("Enter Destination Address: ");
      String ToMail = input.nextLine();
      String ToHost = "smtp.gmail.com";
```

OUTPUT – SIMPLE MESSAGE THROUGH SMTP

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_07>javac -cp "C:\Users\Yuvaraj\Java\javax.mail.jar;C:\Users\Yuvaraj\Java\activation-1.1.1.jar" SMTPMessage
.java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_07>java -cp ".;C:\Users\Yuvaraj\Java\javax.mail.jar;C:\Users\Yuvaraj\Java\activation-1.1.1.jar" SMTPMessag
e
Enter Password for Authentication: barx apvm qkbj rlug
Enter Destination Address: mityuvaraj79@gmail.com
Enter Subject for mail: SMTP - Message
Body:
This email is sent to test SMTP - Simple Mail Transfer Protocol. It is just a simple message.
The Message was sent successfully...

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_07>
```



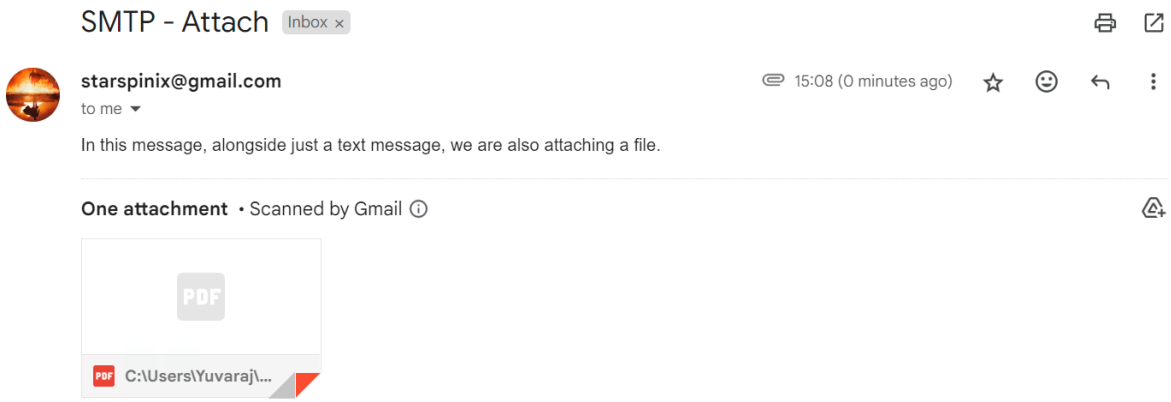
```
        Properties SessionProperties = new Properties();
        SessionProperties.put("mail.smtp.auth","true");
        SessionProperties.put("mail.smtp.starttls.enable","true");
        SessionProperties.put("mail.smtp.host",ToHost);
        SessionProperties.put("mail.smtp.port",587);
        Session CurrentSession = Session.getInstance (
            SessionProperties, new javax.mail.Authenticator() {
                protected PasswordAuthentication getPasswordAuthentication() {
                    return new PasswordAuthentication(SenderMail, SenderPassword);
                }
            }
        );
        try {
            Message ThisMessage = new MimeMessage(CurrentSession);
            ThisMessage.setFrom(new InternetAddress(SenderMail));
            ThisMessage.setRecipients(Message.RecipientType.TO,
InternetAddress.parse(ToMail));
            System.out.print("Enter Subject for mail: ");
            String Subject = input.nextLine();
            System.out.println("Body: ");
            String Body = input.nextLine();
            ThisMessage.setSubject(Subject);
            ThisMessage.setContent(Body,"text/html");
            Transport.send(ThisMessage);
            System.out.println("The Message was sent successfully...");
        }
        catch(Exception e) {
            e.printStackTrace();
        }
        input.close();
    }
}
```

SOURCE CODE – ATTACHING A FILE THROUGH SMTP

```
import javax.mail.*;
import javax.mail.internet.*;
import java.util.*;
import javax.activation.*;

public class SMTPAttach {
   public static void main (String[] args) {
      Scanner input = new Scanner(System.in);
      String SenderUser = "Starspinix";
      String SenderMail = "starspinix@gmail.com";
      System.out.print("Enter Password for Authentication: ");
      String SenderPassword = input.nextLine();
      System.out.print("Enter Destination Address: ");
      String ToMail = input.nextLine();
      String ToHost = "smtp.gmail.com";
      Properties SessionProperties = new Properties();
      SessionProperties.put("mail.smtp.auth","true");
      SessionProperties.put("mail.smtp.starttls.enable","true");
      SessionProperties.put("mail.smtp.host",ToHost);
      SessionProperties.put("mail.smtp.port",587);
      Session CurrentSession = Session.getInstance(
      SessionProperties, new javax.mail.Authenticator() {
         protected PasswordAuthentication
            getPasswordAuthentication() {
               return new PasswordAuthentication (SenderMail,
SenderPassword);
            }
         }
      );
      try {
         Message ThisMessage = new MimeMessage(CurrentSession);
         ThisMessage.setFrom(new InternetAddress(SenderMail));
         ThisMessage.setRecipients(Message.RecipientType.TO,
InternetAddress.parse(ToMail));
         System.out.print("Enter Subject for mail: ");
         String Subject = input.nextLine();
         System.out.println("Body: ");
         String Body = input.nextLine();
```

OUTPUT – ATTACHING A FILE THROUGH SMTP

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_07>javac -cp "C:\Users\Yuvaraj\Java\javax.mail.jar;C:\Users\Yuvaraj\Java\activation-1.1.1.jar" SMTPAttach.
java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_07>java -cp ".;C:\Users\Yuvaraj\Java\javax.mail.jar;C:\Users\Yuvaraj\Java\activation-1.1.1.jar" SMTPAttach
Enter Password for Authentication: barx apvm qkbj rlug
Enter Destination Address: mityuvaraj79@gmail.com
Enter Subject for mail: SMTP - Attach
Body:
In this message, alongside just a text message, we are also attaching a file.
Do you want to add attachment? (y/n): y
Enter Filename to Attach: C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\CN_Lab.pdf
The Message was sent successfully...

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_07>
```



```
            System.out.print("Do you want to add attachment? (y/n): ");
            String c = input.nextLine();
            if(c.equalsIgnoreCase("y")){ System.out.print("Enter Filename to
Attach: ");
                String FileName = input.nextLine();
                DataSource FileSource = new FileDataSource(FileName);
                MimeBodyPart PartOne = new MimeBodyPart();
                PartOne.setText(Body);
                MimeBodyPart PartTwo = new MimeBodyPart();
                PartTwo.setDataHandler(new DataHandler(FileSource));
                PartTwo.setFileName(FileName);
                Multipart MessageBody = new MimeMultipart();
                MessageBody.addBodyPart(PartOne);
                MessageBody.addBodyPart(PartTwo);
                ThisMessage.setContent(MessageBody);
            } else ThisMessage.setContent(Body,"text/html");
            ThisMessage.setSubject(Subject);
            Transport.send(ThisMessage);
            System.out.println("The Message was sent successfully...");
        }
        catch(Exception e) {
            e.printStackTrace();
        }
        input.close();
    }
}
```

RESULT

Hence, SMTP was implemented using Java to send a simple email and an email with a file attachment.

EXP NO: 8	POST OFFICE PROTOCOL
09/09/2024	

AIM

To create a program in Java that implements POP3 from a client to a POP3 compatible server in order to go through received emails.

ALGORITHM

1) Set up the properties for the mail session.
2) Create a javax.mail authenticator object.
3) Create a mail session.
4) Get POP3 store provider and connect to the store of the email.
5) Retrieve messages from the Inbox folder.
6) Listen until the user enters ‘STOP’.

SOURCE CODE

```
import javax.mail.*;
import java.util.*;
import javax.activation.*;
import javax.mail.internet.*;

public class POP3 {
   public static void main(String[] args) throws Exception {
      Scanner input = new Scanner(System.in);
      final String UserName = "starspinix@gmail.com";
      System.out.print("Enter Password: ");
      String Password = input.nextLine();
      Properties SessionProperties = new Properties();
      SessionProperties.put("mail.pop3.socketFactory.class",
"javax.net.ssl.SSLSocketFactory");
      SessionProperties.put("mail.pop3.socketFactory.fallback", "false");
      SessionProperties.put("mail.pop3.socketFactory.port", "995");
      SessionProperties.put("mail.pop3.port", "995");
      SessionProperties.put("mail.pop3.host", "pop.gmail.com");
```

OUTPUT

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_08>javac -cp "C:\Users\Yuvaraj\Java\javax.mail.jar;C:\Users\Yuvaraj\Java\activation-1.1.1.jar" POP3.java

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_08>java -cp ".;C:\Users\Yuvaraj\Java\javax.mail.jar;C:\Users\Yuvaraj\Java\activation-1.1.1.jar" POP3
Enter Password: birw zvoo axza uldf
Delivered-To: mityuvaraj79@gmail.com
Received: by 2002:a05:6359:459f:b0:1c2:ef3f:b9b2 with SMTP id no31csp1235605rwb;
        Mon, 25 Nov 2024 01:35:11 -0800 (PST)
X-Received: by 2002:a05:6a20:72a5:b0:1d5:1354:5256 with SMTP id adf61e73a8af0-1e09e5cbe51mr18340543637.39.1732527310875;
        Mon, 25 Nov 2024 01:35:10 -0800 (PST)
ARC-Seal: i=1; a=rsa-sha256; t=1732527310; cv=none;
        d=google.com; s=arc-20240605;
        b=JP74p+UAgR7J32Dxb6m1z8n8h94THBYw9Mv9ercOrJpCQDefYbRvIw9/AS9VnvYXCR
         myiFLzUnR8g/Obt3rO9lyCq8S5zxT0Of/adhneYD2tgWaeudKpCZ1FOhJKrPc/fS1WcW
         of3d6YEDQqN2aWudw/ZdJKZaZbqi5INp0r3Xgu87RER2FypujBAjq5ucWszdgJ7wLRt6
         0g1zf3Gsv4bl1z62fp8pwgTkh9bS5dJbPPuOp6RsUxVmmZ1XeSDgvGmdab/iNwhFLpZM
         V/8IukBD0u+sj9VNJxz3hmPYLLDOGbfStAI9gpdVXDiRztDZN95HR9nkX4STgogCjbJZ
         qQWQ==
ARC-Message-Signature: i=1; a=rsa-sha256; c=relaxed/relaxed; d=google.com; s=arc-20240605;
        h=content-transfer-encoding:mime-version:subject:message-id:to:from
         :date:dkim-signature;
        bh=I0cm2qApE5qaCwc77deCG6NgnTzGGCgSbKHaOLP6NZ0=;
        fh=+/HYBb5CEDnIwwzKp+aqT9t2HkppFRZpzf8b25X752o=;
        b=CHY0SUg8F+M53j8RHvBoAd2tNgeQ/2JieLWL0W0l8DOnIOvXih2MyuKKmSFV+o7SjL
         T5M4SGeNSj5+3ASWXX2QTQuw81YwLeuxEeAhqy5idWGmbFabqDYjavXH4w2b3JiWdKPV
         aFvkPvciXuxiNNhcU+6e4A7lGW5o+2PHIbtqXD1hVWSn8XZDcaJ87Ufjwssgna5NVrOi
         01AvNTELdcgJ3q4YcZwOS6EJO6spLRzfS2MVBC2qOTTX73+bGsia8yKa/g2FWwX1GMAZ
         BFYn/Er8JYvNo3a89WOS0D7DkvnmHji1/3TVpw5YIh/uOMMeX/3Z74NjNDS6HWzGo2kd
         hEgg==;
        dara=google.com
ARC-Authentication-Results: i=1; mx.google.com;
       dkim=pass header.i=@gmail.com header.s=20230601 header.b=YyxWX3ek;
       spf=pass (google.com: domain of starspinix@gmail.com designates 209.85.220.41 as permitted sender) smtp.mailfrom=starspinix@gmail.com;
       dmarc=pass (p=NONE sp=QUARANTINE dis=NONE) header.from=gmail.com;
       dara=pass header.i=@gmail.com
Return-Path: <starspinix@gmail.com>
Received: from mail-sor-f41.google.com (mail-sor-f41.google.com. [209.85.220.41])
        by mx.google.com with SMTPS id 98e67ed59e1d1-2eb0d0827f0sor4717996a91.0.2024.11.25.01.35.10
        for <mityuvaraj79@gmail.com>
        (Google Transport Security);
        Mon, 25 Nov 2024 01:35:10 -0800 (PST)
```

```
Received-SPF: pass (google.com: domain of starspinix@gmail.com designates 209.85.220.41 as permitted sender) client-ip=209.85.220.41;
Authentication-Results: mx.google.com;
       dkim=pass header.i=@gmail.com header.s=20230601 header.b=YyxWX3ek;
       spf=pass (google.com: domain of starspinix@gmail.com designates 209.85.220.41 as permitted sender) smtp.mailfrom=starspinix@gmail.com;
       dmarc=pass (p=NONE sp=QUARANTINE dis=NONE) header.from=gmail.com;
       dara=pass header.i=@gmail.com
DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed;
        d=gmail.com; s=20230601; t=1732527310; x=1733132110; dara=google.com;
        h=content-transfer-encoding:mime-version:subject:message-id:to:from
         :date:from:to:cc:subject:date:message-id:reply-to;
        bh=I0cm2qApE5qaCwc77deCG6NgnTzGGCgSbKHaOLP6NZ0=;
        b=YyxWX3ekZLw4UIXH2kvAq+p0T4sW4UzS2mlJJE/ZElYFgijihvKZgqiwWpbjMZKzdo
         VliEzCthllx4Ryg0B/cI1eTqoLBeAnVj95hulyra6n3QtOz9zS0OOqSyu/jkQhFXyJY6
         tfS+Q+CAS6uGGNaxCL01g8sPiIO9j45+pbgIfWtNN3QH2CKBM9BXcwTe28/BfUlg93ps
         ZnbzEviBsR+YyO+Ykig/yPBKAMRmqFXJ44ni52Jt926fMlNuMS63kI/ARANPxdtV6GMO
         rbeH5dSZkng5Dbm1FbUuFDdj9wd0KA+YNuIr4g0J8O2nukmc5r5aEF32x9YjFPdpfz6/
         0nGQ==
X-Google-DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/relaxed;
        d=1e100.net; s=20230601; t=1732527310; x=1733132110;
        h=content-transfer-encoding:mime-version:subject:message-id:to:from
         :date:x-gm-message-state:from:to:cc:subject:date:message-id:reply-to;
        bh=I0cm2qApE5qaCwc77deCG6NgnTzGGCgSbKHaOLP6NZ0=;
        b=VOB8TeXuWK5zTt+1UOFfZVuJT6BbthGCIp01Ba/a2kJWUymhTLoZ1enjR2MnTwc8Bg
         X9ma8zoaSROt3Q+srxIMoWT7+1LIICVBUUdD7rqVPHr0Q5WeZyyPvDnMYUDRVbDNlNRf
         5VOqA5RMTGX9bhMKpCKKX1hahtKD/s7ENz5YavO0PRHwpIJ6WPF7tTqWYluXkm7uO0Pc
         Wg1hDRYD2bRvWuXtPWY5kUpkct00t1Q/fPsy8SbrBnv4cTqA+7cOUBgt1gIONgEQBKmx
         OEzb7uUyFN/+r/VPqG1MuAH4PP1kt/oDb9ODu96SBXl8f5yFvojadkmU6CF0Og1EmMpB
         HP/Q==
X-Gm-Message-State: AOJu0YyC3GzYdCIDKXypckeZma7IOs/Wp5aeduJNYFY2njU4OF5lwL42
        MrcGZCiAHroHZWiEBCj8+y+ojZPAkUdRzpdBNZKI2w5m+wl+97hk9GBUbw==
X-Gm-Gg: ASbGncvqWWw7Cge3KGBtRvw+iK+oHOG+bVSoc3ykFvRi33BLYl4Ntke5xXQX24WTCEw
        PyI2ddRg0fg0ezgpTBWdVqqod1/6G/4Gpi2eJ2RnkH9zM5dmoU+lH4qmgxGSEHiuAJTc+iCBDjy
        ykJ8wkKRvLkHN3EpFERB60qUqksq16aax5CRNcdcYzb7iezTMR6qusxC9pWsLjflPwLVvMD1zVe
        sZBfz/Xij/es2Jc+IJq0I/J7FPPAe0KEO51EE/1YM8bz9hMFMJUay7RJ+nodw==
X-Google-Smtp-Source: AGHT+IEVXkAyCEShJyYLR5sY4ICvAx2guhThf8qLtKawRDmlHqp0+IKrhaTzPI4bwzxZLoa0u9VIkA==
X-Received: by 2002:a17:90b:4a09:b0:2ea:9309:75a7 with SMTP id 98e67ed59e1d1-2eb0e125456mr15764387a91.2.1732527309911;
        Mon, 25 Nov 2024 01:35:09 -0800 (PST)
Return-Path: <starspinix@gmail.com>
Received: from 192.168.1.8 ([103.78.164.230])
        by smtp.gmail.com with ESMTPSA id 98e67ed59e1d1-2eaf0c41e40sm7991461a91.44.2024.11.25.01.35.08
        for <mityuvaraj79@gmail.com>
```

```
        (version=TLS1_3 cipher=TLS_AES_256_GCM_SHA384 bits=256/256);
        Mon, 25 Nov 2024 01:35:09 -0800 (PST)
Date: Mon, 25 Nov 2024 15:05:05 +0530 (IST)
From: starspinix@gmail.com
To: mityuvaraj79@gmail.com
Message-ID: <105704967.0.1732527305644@[192.168.1.8]>
Subject: SMTP - Message
MIME-Version: 1.0
Content-Type: text/html; charset=us-ascii
Content-Transfer-Encoding: 7bit

This email is sent to test SMTP - Simple Mail Transfer Protocol. It is just a simple message.

Do you want to continue? (yes/no): no
Mails read successfully. Exiting...

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_08>
```

```
        SessionProperties.put("mail.pop3.user", UserName);
        SessionProperties.put("mail.store.protocol", "pop3");
        Session ThisSession = Session.getDefaultInstance (
            SessionProperties,
            new javax.mail.Authenticator() {
                protected PasswordAuthentication getPasswordAuthentication() {
                    return new PasswordAuthentication(UserName, Password);
                }
            }
        );
        Store MailServerStore = ThisSession.getStore("pop3");
        MailServerStore.connect("pop.gmail.com", UserName, Password);
        Folder Inbox = MailServerStore.getFolder("INBOX");
        Inbox.open(Folder.READ_ONLY);
        Message[] MailMessages = Inbox.getMessages();
        for (Message mail : MailMessages) {
            mail.writeTo(System.out);
            System.out.print("\nDo you want to continue? (yes/no): ");
            String Choice = input.nextLine();
            if (Choice.equalsIgnoreCase("no")) {
                System.out.println("Mails read successfully. Exiting...");
                break;
            }
        }
        Inbox.close();
        MailServerStore.close();
        input.close();
    }
}
```

RESULT

Hence, POP3 was successfully implemented in a Java program to read the Inbox of the given email.

EXP NO: 9	PING COMMAND
16/09/2024	

AIM

To write a program in Java that implements the ping command in raw sockets.

ALGORITHM

1) Create a raw socket in the client program.
2) Get the name of the host whose IP address is to be resolved using ICMP.
3) Pass this name to the ICMP server through the raw socket.
4) The server will respond with the IP address of the host.
5) Receive the response and display it.

SOURCE CODE – PING SERVER

```
import java.io.*;
import java.net.*;
import java.util.*;

class PingServer {
   public static void main(String[] args) {
      try {
         ServerSocket ss=new ServerSocket(2156);
         Socket s=ss.accept();
         if(s.isConnected())
            System.out.println("Connected ...");
         System.out.println("Listening ...");
         DataInputStream dis=new DataInputStream(s.getInputStream());
         DataOutputStream dos=new DataOutputStream(s.getOutputStream());
         int no=0;
         String ip="";
         if((dis.readUTF()).equals("P")) {
            no=dis.readInt();
            System.out.println("No. of Packets: "+no);
```

```
            }
            if((dis.readUTF()).equals("A")) {
                ip=dis.readUTF();
                System.out.println("Address to Ping: "+ip);
            }
            System.out.println("Running ping -n "+no+" "+ip);
            Process p=Runtime.getRuntime().exec("ping -n "+no+" "+ip);
            BufferedReader br=new BufferedReader(new
InputStreamReader(p.getInputStream()));
            String ipline;
            while((ipline=br.readLine())!=null) {
                dos.writeUTF(ipline);
            }
            dis.close();
            dos.close();
        } catch(Exception x) {
            x.printStackTrace();
        }
    }
}
```

SOURCE CODE – PING CLIENT

```
import java.io.*;
import java.net.*;
import java.util.*;

public class PingClient {
    public static void main(String[] args) {
        try {
            Socket s=new Socket("localhost",2156);
            BufferedReader br=new BufferedReader(new
InputStreamReader(System.in));
            if(s.isConnected())
                System.out.println("Connected!");
            Scanner input=new Scanner(System.in);
            DataInputStream is=new DataInputStream(s.getInputStream());
            DataOutputStream os=new DataOutputStream(s.getOutputStream());
            System.out.print("No. of Packets to Send: ");
            int no=input.nextInt();
```

OUTPUT – PING SERVER

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_09>java PingServer
Connected ...
Listening ...
No. of Packets: 5
Address to Ping: google.com
Running ping -n 5 google.com

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_09>
```

OUTPUT – PING CLIENT

```
C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_09>java PingClient
Connected!
No. of Packets to Send: 5
Address to be pinged: google.com

Pinging google.com [142.250.183.238] with 32 bytes of data:
Reply from 142.250.183.238: bytes=32 time=6ms TTL=119
Reply from 142.250.183.238: bytes=32 time=8ms TTL=119
Reply from 142.250.183.238: bytes=32 time=5ms TTL=119
Reply from 142.250.183.238: bytes=32 time=5ms TTL=119
Reply from 142.250.183.238: bytes=32 time=5ms TTL=119

Ping statistics for 142.250.183.238:
    Packets: Sent = 5, Received = 5, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 5ms, Maximum = 8ms, Average = 5ms

End.

C:\Users\Yuvaraj\Desktop\Studies\MIT\5\CN\Lab\exp_09>
```

```
            System.out.print("Address to be pinged: ");
            String ip=br.readLine();
            os.writeUTF("P");
            os.writeInt(no);
            os.writeUTF("A");
            os.writeUTF(ip);
            String pingline;
            while((pingline=is.readUTF())!=null) {
                System.out.println(pingline);
            }
            os.flush();
            os.close();
            is.close();
        } catch(Exception x) {
            System.out.println("\nEnd.");
        }
    }
}
```

RESULT

Hence, ping command was implemented using raw sockets in Java programs.

CYCLE 2

OUTPUT – variables.tcl

```
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$ ns variables.tcl
10
a = 10
b = 15
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$
```

OUTPUT – expressions.tcl

```
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$ ns expressions.tcl
c = 25
d = 125
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$
```

OUTPUT – control.tcl

```
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$ ns control.tcl
10
9
8
7
6
5
4
3
2
1
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$
```

OUTPUT – files.tcl

```
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$ ls
arrays.tcl  control.tcl  expressions.tcl  files.tcl  procedure.tcl  variables.tcl
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$ ns files.tcl
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$ ls
arrays.tcl  control.tcl  expressions.tcl  f1.txt  files.tcl  procedure.tcl  variables.tcl
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$ cat f1.txt
Test Message
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$
```

EXP NO: 1	STUDY OF NS-2
30/09/2024	

AIM

To study network simulator version 2 (ns2).

SOURCE CODE – variables.tcl

```
set a 10
set b 15
puts $a
puts "a = $a"
puts "b = $b"
```

SOURCE CODE – expressions.tcl

```
set a 10
set b 15
set c [expr $a + $b]
puts "c = $c"
set d [expr [expr $b - $a] * $c]
puts "d = $d"
```

SOURCE CODE – control.tcl

```
set x 10
while { $x > 0 } {
	puts $x
	set x [expr $x - 1]
}
```

SOURCE CODE – files.tcl

```
set test [open f1.txt w]
puts $test "Test Message"
```

OUTPUT – procedure.tcl

```
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$ ns procedure.tcl
Test Message
10 + 20 = 30
node_(0): n0
node_(1): n1
node_(2): n2
node_(3): n3
node_(4): n4
node_(5): n5
node_(6): n6
node_(7): n7
node_(8): n8
node_(9): n9
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$
```

OUTPUT – arrays.tcl

```
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$ ns arrays.tcl
1 2
3 4
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_01$
```

SOURCE CODE – procedure.tcl

```
proc display { } {
	puts "Test Message"
}
display
proc add { x y } {
	set z [expr $x + $y]
	puts "$x + $y = $z"
}
add 10 20
proc print { k } {
	for { set i 0 } { $i < $k } { incr i } {
		puts "node_($i): n$i"
	}
}
print 10
```

SOURCE CODE – arrays.tcl

```
set matrix(0,0) 1
set matrix(0,1) 2
set matrix(1,0) 3
set matrix(1,1) 4
set i 0
while { $i < 2 } {
	set j 0
	while { $j < 2 } {
		puts -nonewline $matrix($i,$j)
		puts -nonewline " "
		set j [expr $j + 1]
	}
	puts ""
	set i [expr $i + 1]
}
```

RESULT

Hence, a basic study of network simulator version 2 was done.

EXP NO: 2	STUDY OF NS-2
30/09/2024	

AIM

To generate a network topology of nodes with suitable link characteristics using network simulator version 2.

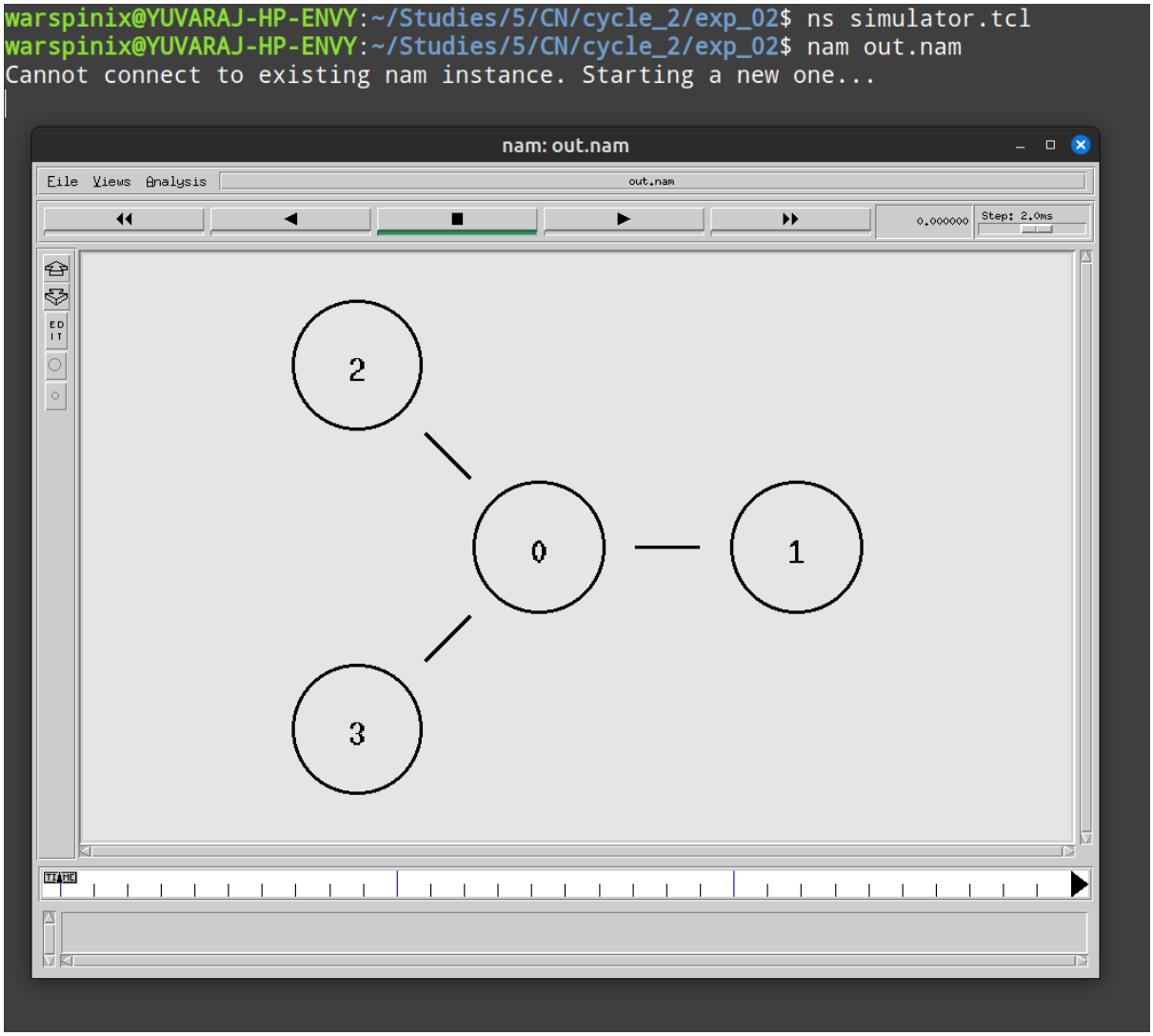
ALGORITHM

1) Create a simulator object which is an event scheduler using set ns [new Simulator].
2) Turn on tracing by opening the NAM trace file in writing mode and trace all the packets.
3) Create 4 topological nodes and assign corresponding variables.
4) Create duplex links between nodes (n0, n1), (n2, n0), and (n3, n0).
5) Set the orientation of the nodes.
6) Run the network simulator using the command $ns run.

SOURCE CODE

```
set ns [new Simulator]

set tr [open out.tr w]
$ns trace-all $tr

set namtr [open out.nam w]
$ns namtrace-all $namtr

set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]

$ns duplex-link $n0 $n1 10Mb 5ms DropTail
$ns duplex-link $n2 $n0 10Mb 5ms DropTail
$ns duplex-link $n3 $n0 10mb 5ms DropTail
```

OUTPUT


warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_02$ ns simulator.tcl
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_02$ nam out.nam
Cannot connect to existing nam instance. Starting a new one...
nam: out.nam
File Views Analysis
out.nam
0.000000
Step: 2.0ms
2
0
1
3
TIME

```
$ns duplex-link-op $n0 $n1 orient right
$ns duplex-link-op $n0 $n2 orient left-up
$ns duplex-link-op $n0 $n3 orient left-down
$ns at 10.0 "$ns halt"
$ns run
```

RESULT

Hence, a network topology of 4 nodes with suitable links was successfully created using ns2.

EXP NO: 3	WIRED NETWORK USING TCP
07/10/2024	

AIM

To generate a wired network using TCP in ns2.

ALGORITHM

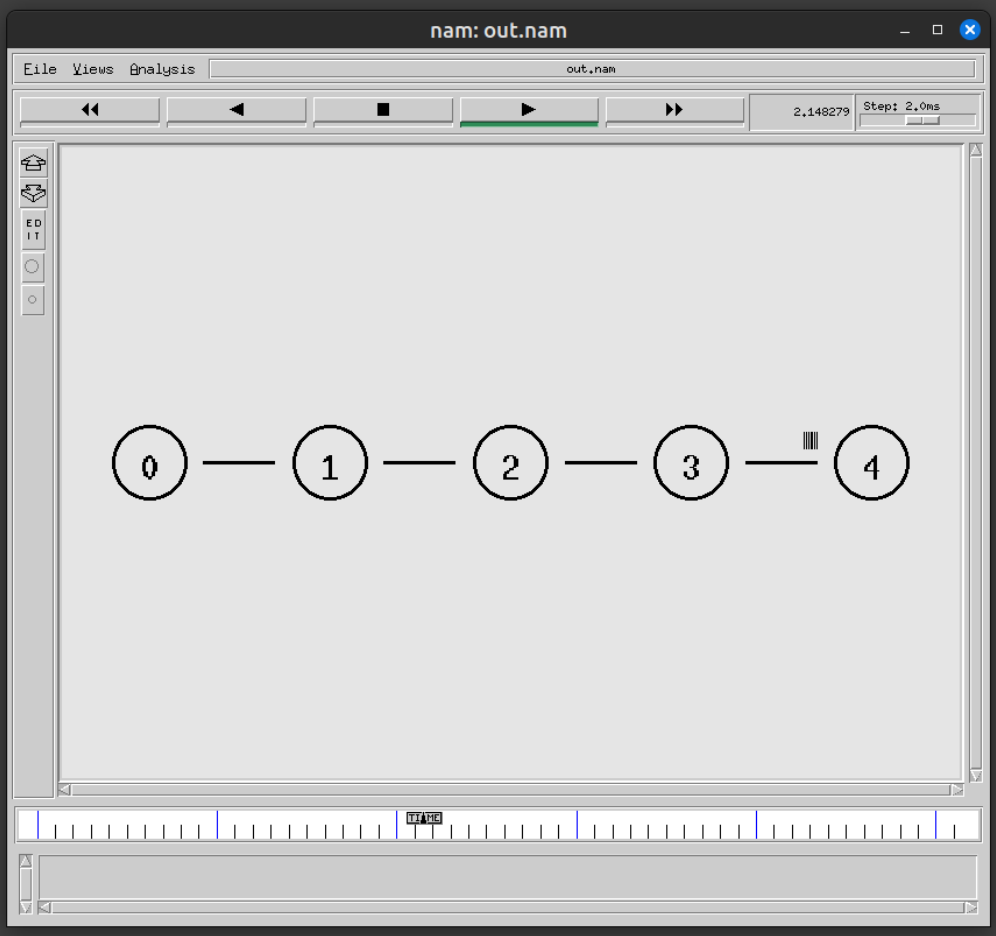
1) Declare a new Simulator ns.
2) Define a procedure finish to execute the namfile.
3) Create five nodes n0, n1, n2, n3, and n4 with duplex links.
4) Set the orientation of the nodes.
5) Set up a TCP Agent and create an FTP application.
6) Start the traffic and call the finish procedure, which finishes at 5.5 seconds.
7) Run the simulator.

SOURCE CODE

```
set ns [new Simulator]

set namtr [open out.nam w]
$ns namtrace-all $namtr

set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]

$ns duplex-link $n0 $n1 10MB 10ms DropTail
$ns duplex-link $n1 $n2 20MB 10ms DropTail
$ns duplex-link $n2 $n3 10MB 10ms DropTail
$ns duplex-link $n3 $n4 20MB 10ms DropTail

$ns duplex-link-op $n0 $n1 orient right
$ns duplex-link-op $n1 $n2 orient right
```

OUTPUT

```
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_03$ ns simulator.tcl
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_03$ nam out.nam
Cannot connect to existing nam instance. Starting a new one...
```



```
$ns duplex-link-op $n2 $n3 orient right
$ns duplex-link-op $n3 $n4 orient right

set tcp [new Agent/TCP]
$ns attach-agent $n0 $tcp

set sink [new Agent/TCPSink]
$ns attach-agent $n4 $sink
$ns connect $tcp $sink

set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP
$ftp set packet-size_ 1000

proc finish {} {
   global ns namtr
   $ns flush-trace
   close $namtr
   exec nam out.nam &amp;amp;
   exit 0
}

$ns at 0.1 "$ftp start";
$ns at 5.0 "$ftp stop";
$ns at 5.5 "finish";

$ns run
```

RESULT

Hence, a wired network using TCP was successfully simulated in ns2.

EXP NO: 4	WIRELESS NETWORK USING UDP
14/10/2024	

AIM

To generate a wireless network topological simulation using UDP in ns2.

ALGORITHM

1) Initialize necessary variables for wireless connection.
2) Create tracing and animation files.
3) Set temporary, and load the values of the variables.
4) Create nodes and mark their locations.
5) Create a channel, i.e., communication path for the nodes.
6) Specify mobility codes if any of the nodes are moving.
7) Set CBR traffic.
8) Run the simulator.

SOURCE CODE

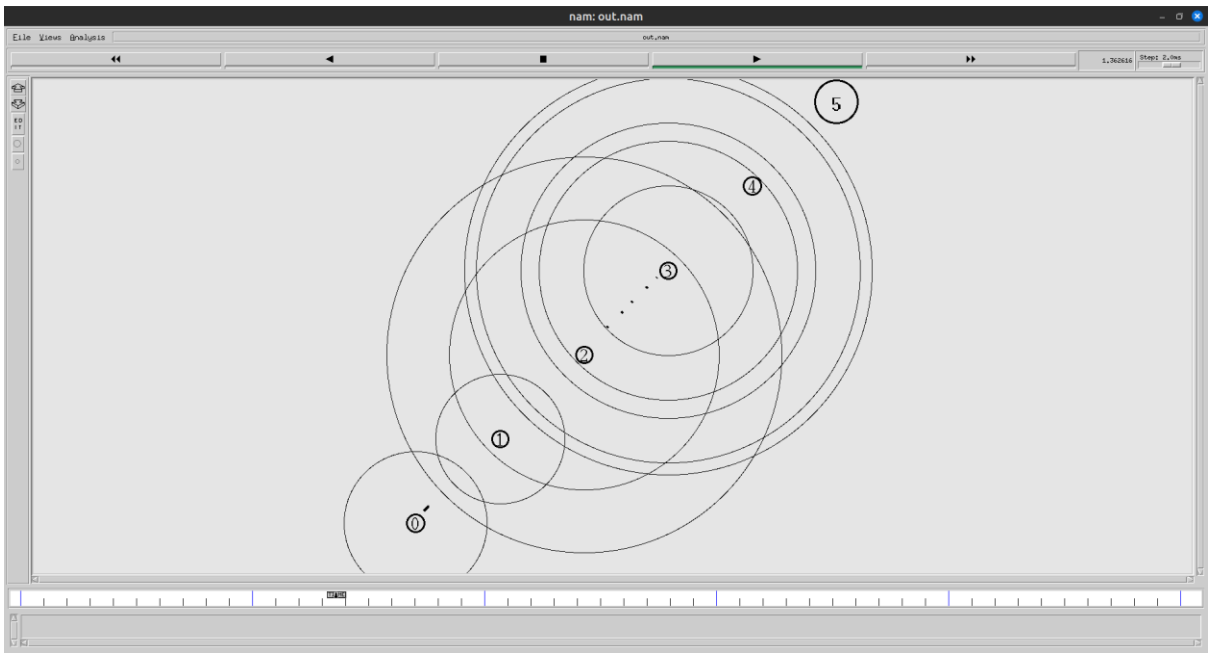
```
set tr [open out.tr w]
$ns trace-all $tr

set namtr [open out.nam w]
$ns namtrace-all-wireless $namtr $val(x) $val(y)

set topo [new Topography]
$topo load_flatgrid $val(x) $val(y)

create-god $val(nn)

set channel1 [new $val(chan)]
set channel2 [new $val(chan)]
set channel3 [new $val(chan)]
```

```
$ns node-config -adhocRouting $val(rp) \
-llType $val(ll) \
-macType $val(mac) \
-ifqType $val(ifq) \
-ifqLen $val(ifqlen) \
-antType $val(ant) \
-propType $val(prop) \
-phyType $val(netif) \
-topoInstance $topo \
-agentTrace ON \
-macTrace ON \
-routerTrace ON \
-movementTrace ON \
-channel $channel1

set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]

$n0 random-motion 0
$n1 random-motion 0
$n2 random-motion 0
$n3 random-motion 0
$n4 random-motion 0
$n5 random-motion 0

$ns initial_node_pos $n0 20
$ns initial_node_pos $n1 20
$ns initial_node_pos $n2 20
$ns initial_node_pos $n3 20
$ns initial_node_pos $n4 20
$ns initial_node_pos $n5 50

$n0 set X_ 0.0
$n0 set Y_ 0.0
$n0 set Z_ 0.0
```

```
$n1 set X_ 100.0
$n1 set Y_ 100.0
$n1 set Z_ 0.0

$n2 set X_ 200.0
$n2 set Y_ 200.0
$n2 set Z_ 0.0

$n3 set X_ 300.0
$n3 set Y_ 300.0
$n3 set Z_ 0.0

$n4 set X_ 400.0
$n4 set Y_ 400.0
$n4 set Z_ 0.0

$n5 set X_ 500.0
$n5 set Y_ 500.0
$n5 set Z_ 0.0

$ns at 0.0 "$n0 setdest 10.0 10.0 0.0"
$ns at 0.0 "$n1 setdest 100.0 100.0 0.0"
$ns at 0.0 "$n2 setdest 200.0 200.0 0.0"
$ns at 0.0 "$n3 setdest 300.0 300.0 0.0"
$ns at 0.0 "$n4 setdest 400.0 400.0 0.0"
$ns at 0.0 "$n5 setdest 499.0 499.0 0.0"
$ns at 3.0 "$n5 setdest 480.0 480.0 0.0"

set tcp [new Agent/TCP]
set sink [new Agent/TCPSink]
$ns attach-agent $n0 $tcp
$ns attach-agent $n5 $sink
$ns connect $tcp $sink

set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ns at 1.0 "$ftp start"

set udp [new Agent/UDP]
set null [new Agent/Null]
```

OUTPUT

```
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_04$ ns simulator.tcl
num_nodes is set 6
INITIALIZE THE LIST xListHead
Starting Simulation
channel.cc:sendUp - Calc highestAntennaZ_ and distCST_
highestAntennaZ_ = 1.5,  distCST_ = 550.0
SORTING LISTS ...DONE!
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_04$ nam out.nam
```



```
$ns attach-agent $n2 $udp
$ns attach-agent $n3 $null
$ns connect $udp $null

set cbr [new Application/Traffic/CBR]
$cbr attach-agent $udp
$ns at 1.0 "$cbr start"

$ns at 5.0 "finish"

proc finish {} {
   global ns tr namtr
   $ns flush-trace
   close $tr
   close $namtr
   exit 0
}

puts "Starting Simulation"
$ns run
```

RESULT

Hence, a wireless network topological simulation was successfully implemented using UDP in ns2.

EXP NO: 5	PERFORMANCE ANALYSIS USING XGRAPH
21/10/2024	

AIM

To analyse the performance of UDP using XGraph.

ALGORITHM

1) Create a new simulator ns.
2) Create five nodes and establish duplex links with appropriate nodes.
3) Declare a procedure attach-expoo-traffic and define it to simulate a UDP traffic.
4) Create sink agents and attach it with the nodes.
5) Declare and define the finish procedure.
6) Declare a procedure record to track the bytes received by the traffic links.
7) Calculate the bandwidth and write it to the file.
8) Reset the bytes value on the traffic sinks.
9) Reschedule the procedures by calling each defined function in the appropriate order.
10) Run the simulator.

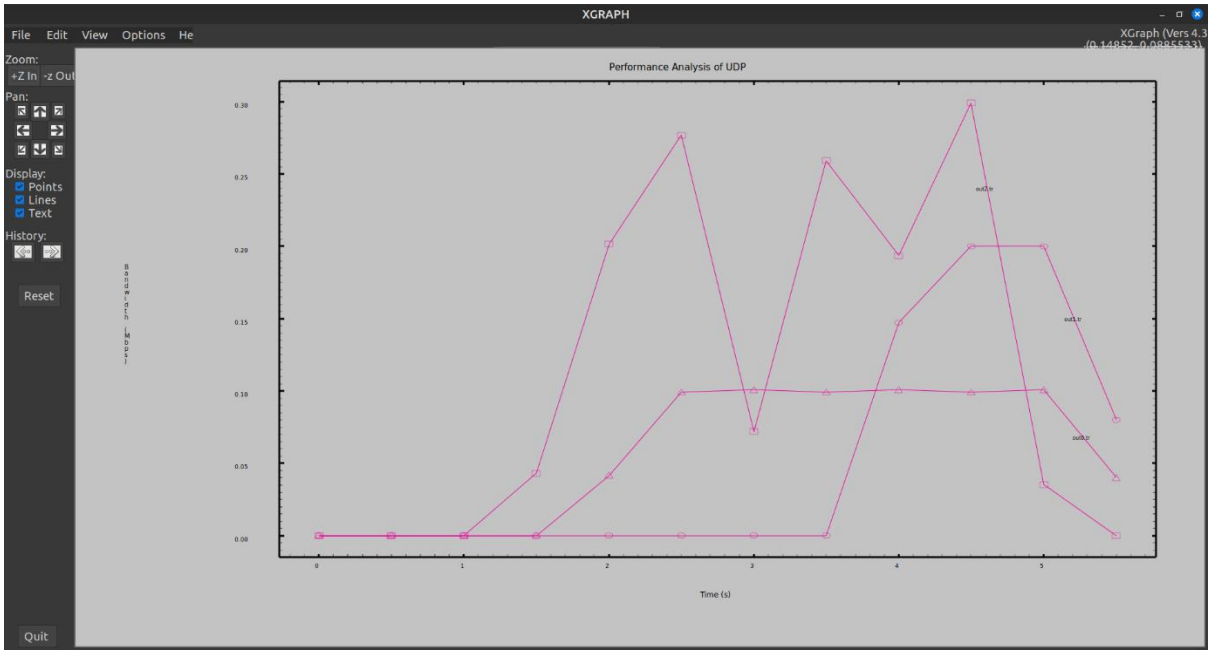
SOURCE CODE

```
set ns [new Simulator]

set namtr [open out.nam w]
$ns namtrace-all $namtr

set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]

$ns duplex-link $n0 $n3 1Mb 100ms DropTail
$ns duplex-link $n1 $n3 1Mb 100ms DropTail
```

OUTPUT

```
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_05$ ns simulator.tcl
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_05$ cat out0.tr
0 0.0
0.5 0.0
1 0.0
1.5 0.0
2 0.041599999999999998
2.5 0.099199999999999997
3 0.1008
3.5 0.099199999999999997
4 0.1008
4.5 0.099199999999999997
5 0.1008
5.5 0.040000000000000001
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_05$ cat out1.tr
0 0.0
0.5 1.5999999999999999e-05
1 1.5999999999999999e-05
1.5 1.5999999999999999e-05
2 1.5999999999999999e-05
2.5 1.5999999999999999e-05
3 1.5999999999999999e-05
3.5 1.5999999999999999e-05
4 0.14721600000000001
4.5 0.200016
5 0.200016
5.5 0.080016000000000004
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_05$ cat out2.tr
0 0.0
0.5 3.1999999999999999e-05
1 3.1999999999999999e-05
1.5 0.043232
2 0.20163200000000001
2.5 0.27683200000000002
3 0.072031999999999999
3.5 0.25923200000000002
4 0.193632
4.5 0.299232
5 0.035231999999999999
5.5 3.1999999999999999e-05
warspinix@YUVARAJ-HP-ENVY:~/Studies/5/CN/cycle_2/exp_05$
```

```
$ns duplex-link $n2 $n3 1Mb 100ms DropTail
$ns duplex-link $n3 $n4 1Mb 100ms DropTail

$ns duplex-link-op $n0 $n3 orient right
$ns duplex-link-op $n1 $n3 orient down
$ns duplex-link-op $n2 $n3 orient left
$ns duplex-link-op $n3 $n4 orient down

proc attach-expoo-traffic { node sink size burst idle rate } {
   set ns [Simulator instance]

   set source [new Agent/UDP]
   $ns attach-agent $node $source

   set traffic [new Application/Traffic/Exponential]
   $traffic set packetSize_ $size
   $traffic set burst_time $burst
   $traffic set idle_time_ $idle
   $traffic set rate_ $rate

   $traffic attach-agent $source
   $ns connect $source $sink
   return $traffic
}

set sink0 [new Agent/LossMonitor]
set sink1 [new Agent/LossMonitor]
set sink2 [new Agent/LossMonitor]

$ns attach-agent $n4 $sink0
$ns attach-agent $n4 $sink1
$ns attach-agent $n4 $sink2

set source0 [attach-expoo-traffic $n0 $sink0 100 2s 1s 100k]
set source1 [attach-expoo-traffic $n1 $sink1 100 2s 1s 200k]
set source2 [attach-expoo-traffic $n2 $sink2 100 2s 1s 300k]

set tr0 [open out0.tr w]
set tr1 [open out1.tr w]
set tr2 [open out2.tr w]
```

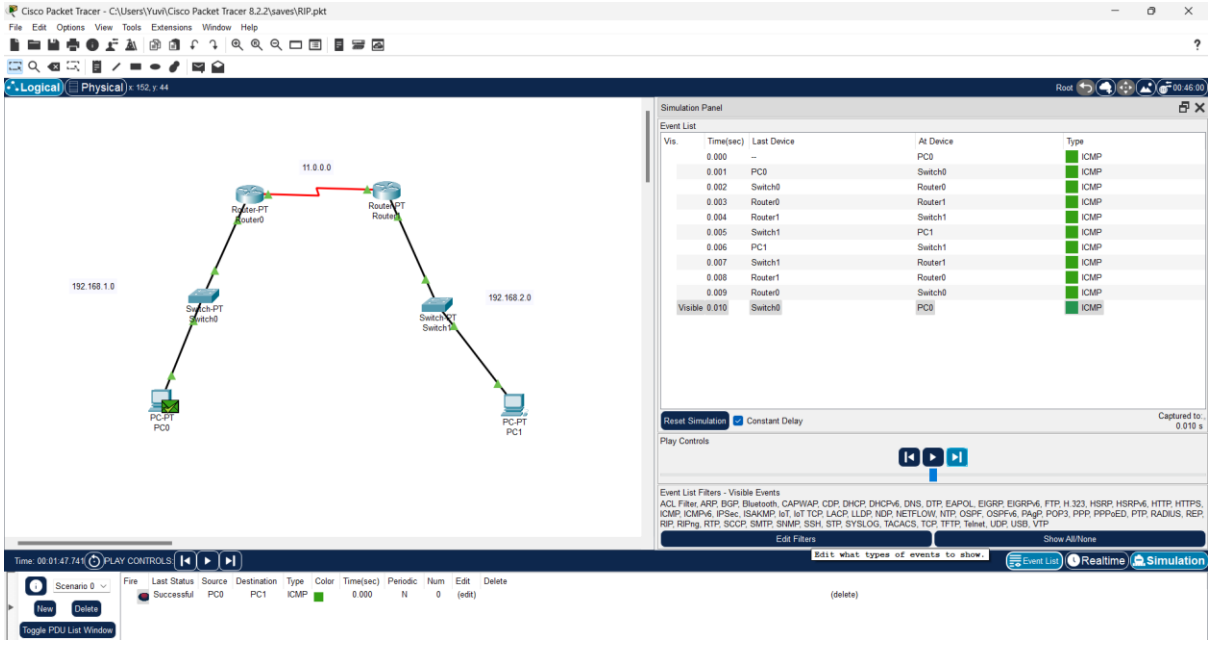

XGRAPH
File
Edit
View
Options
He
XGraph (Vers 4.3
Zoom:
+Z In
-z Out
Pan:
Display:
Points
Lines
Text
History:
Reset
Quit
Performance Analysis of UDP
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
1
2
3
4
5
Time (s)

```
proc finish {} {
   global tr0 tr1 tr2
   global ns namtr
   $ns flush-trace
   close $namtr
   exec nam out.nam &amp;amp;
   close $tr0
   close $tr1
   close $tr2
   exit 0
}

proc record {} {
   global sink0 sink1 sink2 tr0 tr1 tr2

   set ns [Simulator instance]
   set time 0.5
   set bw0 [$sink0 set bytes_]
   set bw1 [$sink1 set bytes_]
   set bw2 [$sink2 set bytes_]
   set now [$ns now]

   puts $tr0 "$now [expr $bw0/$time * 8/1000000]"
   puts $tr1 "$now [expr $bw1/$time * 8/1000000]"
   puts $tr2 "$now [expr $bw2/$time * 8/1000000]"
   $sink0 set bytes_ 0
   $sink1 set bytes_ 1
   $sink2 set bytes_ 2
   $ns at [expr $now+$time] "record"
}

$ns at 0.0 "record"
$ns at 1.0 "$source0 start"
$ns at 1.0 "$source1 start"
$ns at 1.0 "$source2 start"
$ns at 5.0 "$source0 stop"
$ns at 5.0 "$source1 stop"
$ns at 5.0 "$source2 stop"
$ns at 6.0 "finish"
$ns run
```

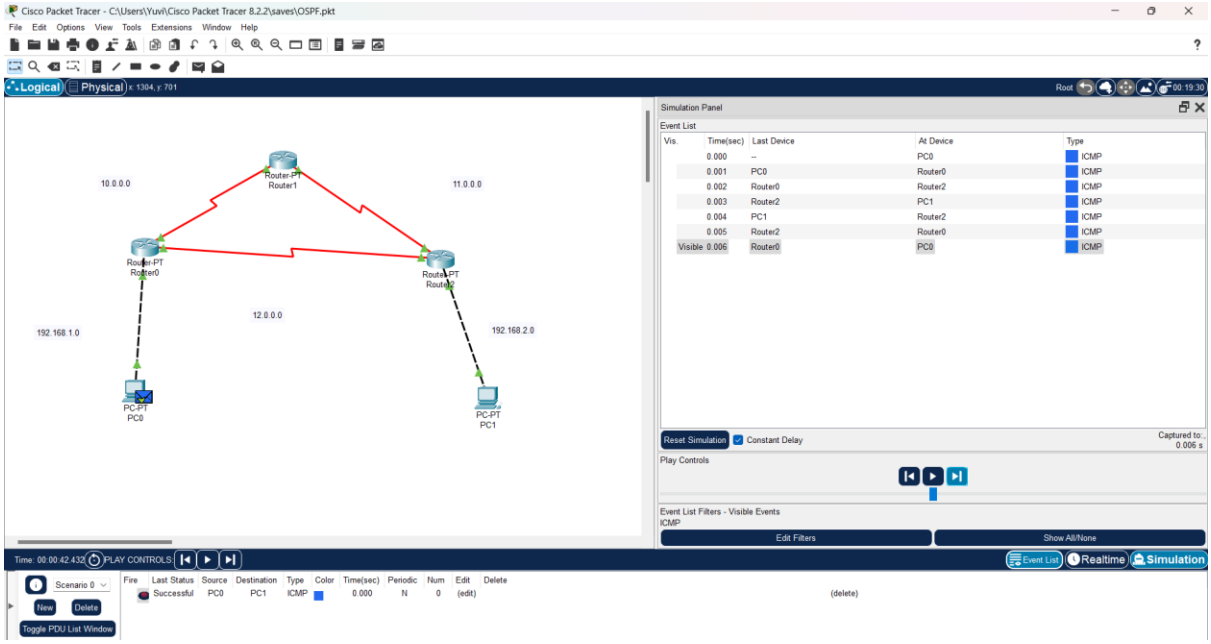
RESULT

Hence, performance analysis of UDP was successfully executed and visualised using XGraph and ns2.

OUTPUT – RIP



OUTPUT – OSPF



EXP NO: 6	RIP AND OSPF
28/10/2024	

AIM

To simulate the working of Routing Information Protocol (RIP) and Open Shortest Path First (OSPF) in Cisco Packet Tracer.

PROCEDURE – RIP

1) Open a new window in Cisco Packet Tracer.
2) Select two end devices (PCs), two network devices (PT routers), and two switches (PT switches), and drag them from the menu.
3) Connect all of them as shown in the diagram.
4) Name the two networks as A and B and assign them a particular IP address.
5) Configure the two end devices using the IP address assigned.
6) Similarly, assign the IP address between the two connected Routers.
7) Send a packet from PC0 to Router0, and similarly from PC1 to Router1.
8) Now try to send the packet from PC0 to PC1. It will fail.
9) Configure the routers and add all the IP addresses to the RIP of routers and save them.
10) Now send the packets from PC0 to PC1.

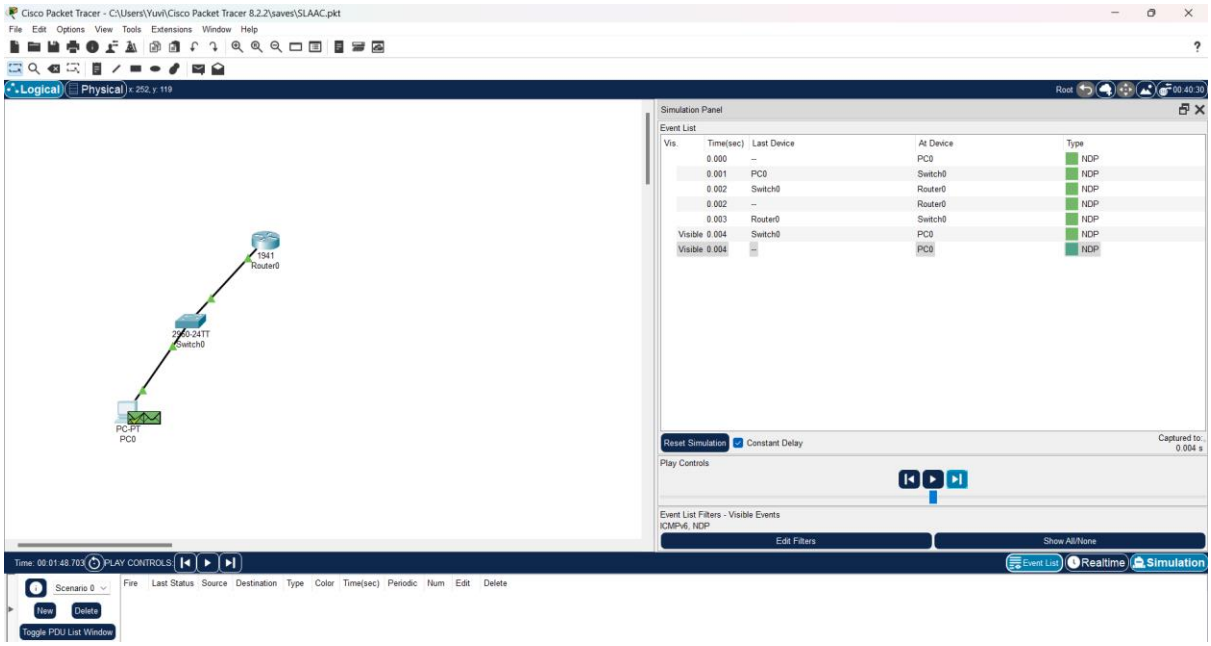
PROCEDURE – OSPF

1) Open a new window in Cisco Packet Tracer.
2) Drag in two PCs and three routers and connect all them.
3) Name the networks with appropriate IP addresses.
4) Configure the PCs and routers to their respective networks.
5) Now use OSPF to make all the networks visible to each other.
6) Use the command line interpreter to activate OSPF.
7) Enable and configure the terminal in the CLI.
8) The OSPF is activated, making all the networks visible to each other.
9) Now send a packet from PC0 to PC1 and vice versa.
10) The routers choose the path to deliver the packet.

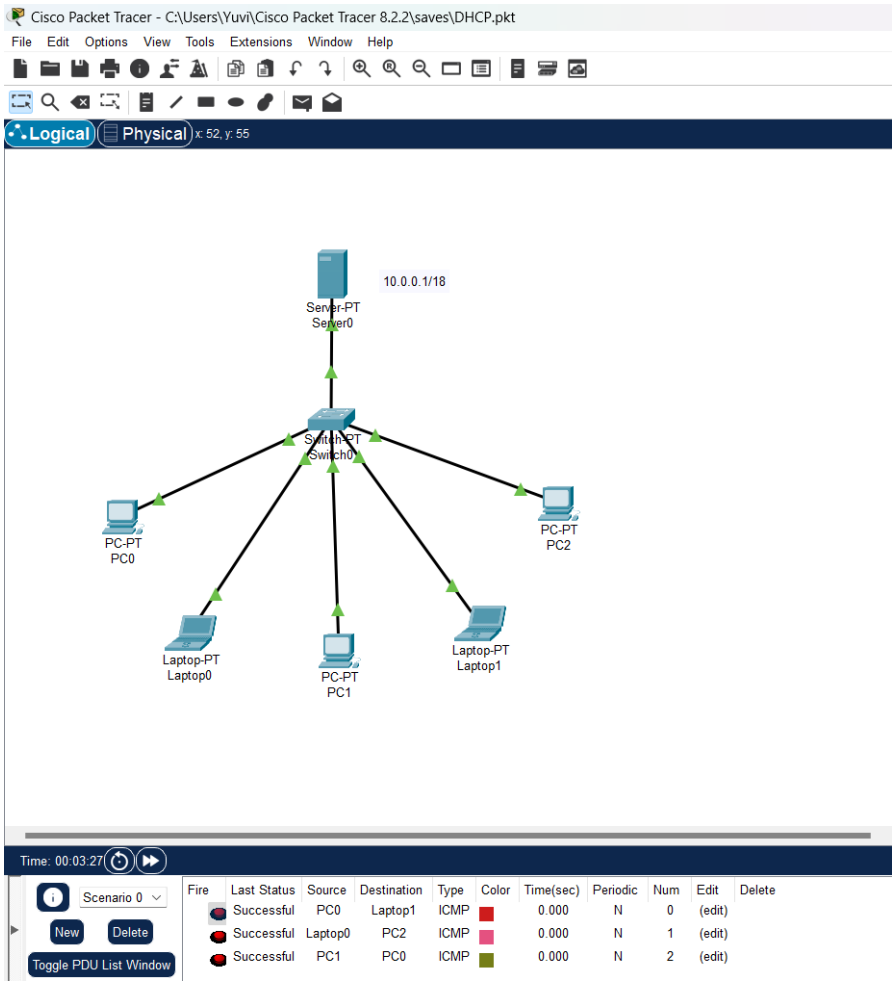
RESULT

Hence, the working of RIP and OSPF were successfully simulated in Cisco Packet Tracer.

OUTPUT – SLAAC



OUTPUT – DHCP



EXP NO: 7	SLAAC AND DHCP
28/10/2024	

AIM

To simulate the working of Stateless Address Autoconfiguration (SLAAC) and Dynamic Host Configuration Protocol (DHCP) in Cisco Packet Tracer.

PROCEDURE – SLAAC

1) Open a new window in Cisco Packet Tracer.
2) Drag and drop a router, a switch, and a PC from the menu bar.
3) Connect the end device with the switch using a fast ethernet cable.
4) Connect the switch to the router using the gigabit ethernet cable.
5) Configure the router and the switch.
6) Set up the IPv6 address for the end system.
7) Initiate an autoconfiguration that kickstarts the Neighbour Discovery Protocol.

PROCEDURE – DHCP

1) Open a new window in Cisco Packet Tracer.
2) Drag and drop a server, a switch, and a few end devices from the menu bar.
3) Connect the end devices to the switch and the switch to the server using the fast ethernet cables.
4) If a connection error occurs, add more ports to the switch.
5) Configure the IP address of the server, and turn DHCP on.
6) For all the end devices, make a DHCP IP address request.
7) Ping one end device from another, or send packets between them to test the connection.

RESULT

Hence, the working of SLAAC and DHCP were successfully simulated in Cisco Packet Tracer.